



Universidad de Castilla-La Mancha
Escuela Superior de Ingeniería Informática

Trabajo Fin de Grado
Grado en Ingeniería Informática
Computación

**Transcripción automática de música de piano a
partitura usando aprendizaje profundo**

María Piedad Castex Sierra

Abril, 2026



TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

Computación

Transcripción automática de música de piano a partitura usando aprendizaje profundo

Autor: María Piedad Castex Sierra

Tutores: Juan Carlos Alfaro Jiménez

José Antonio Gámez Martín

Abril, 2026

*A mi familia,
tanto por sangre como por elección.*

Declaración de autoría

Yo, María Piedad Castex Sierra, con DNI 20514506R, declaro que soy el único autor del trabajo fin de grado titulado “Transcripción automática de música de piano a partitura usando aprendizaje profundo”, que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual, y que todo el material no original contenido en dicho trabajo está apropiadamente atribuido a sus legítimos autores.

Albacete, a ... de ... de 2026

Fdo.: María Piedad Castex Sierra

Resumen

La transcripción automática de música (*Automatic Music Transcription*, AMT) consiste en convertir una señal de audio en una representación simbólica estructurada. En el caso del piano polifónico, la superposición de notas simultáneas, el uso del pedal de resonancia y la amplia gama dinámica del instrumento hacen de esta tarea uno de los problemas abiertos más exigentes dentro de la Recuperación de Información Musical.

El presente Trabajo Fin de Grado diseña, implementa y evalúa *Magdalena v1.0*, un sistema de AMT para piano que formula el problema como una tarea de traducción de secuencia a secuencia. El sistema transforma grabaciones de audio en secuencias de 311 *tokens* que codifican el tono, el inicio, la duración y la dinámica de cada evento musical, mediante una arquitectura Encoder-Decoder asimétrica: el codificador integra el mecanismo de atención dispersa instancia-dependiente *Sparsifiner*, adaptado al dominio del audio de longitud variable; el decodificador emplea la arquitectura Transformer canónica con atención cruzada y decodificación autorregresiva.

El modelo se entrenó sobre el corpus MAESTRO v3.0.0 (≈ 200 horas de interpretaciones de piano clásico) en tres configuraciones experimentales. La evaluación sobre el conjunto de prueba reporta valores de F1 de *onset* de hasta 0.103, demostrando la viabilidad de la arquitectura propuesta dentro de las restricciones de hardware disponibles.

Agradecimientos

Es un grandísimo orgullo para mí cerrar la etapa universitaria con un trabajo como este, pero lo es más haberla pasado y haber conocido a las maravillosos y genuinas personas que tengo la suerte de llamar amigos. Aunque feliz por finalmente conseguir lo que me propuse hace 5 años, no seré capaz de no echar de menos estos años, donde hubo espacio tanto para lo académico como para algunas de las mejores experiencias, historias, amor y risas que me voy a echar en la vida (mención especial para Carlos, Nuria, Emilio, Curtínez, Libanio y Brahim).

Por último, pero no por ello menos importante, gracias a mis padres y mi hermano, que llevan aguantandome desde el glorioso día de mi nacimiento. Sin ellos apoyandome en todo lo que me ha interesado, es muy probable que no estuviese escribiendo esto. Y ahora sé todo lo que me habría perdido.

Por ellos y para ellos, gracias por acompañarme y por hacer del viaje una historia digna de contar.

Índice general

1	Introducción General	1
1.1	Contextualización del área de estudio	1
1.2	Motivación y planteamiento del problema	2
1.3	Objetivos	3
1.4	Contribuciones	4
1.5	Estructura de la memoria	5
2	Estado del arte	7
2.1	Representación del audio para el aprendizaje profundo	7
2.1.1	La señal de audio digital	7
2.1.2	Análisis tiempo-frecuencia: la STFT	8
2.1.3	El espectrograma Mel logarítmico	8
2.2	Transcripción automática de música (AMT)	9
2.2.1	Definición del problema	9
2.2.2	Representaciones habituales	10
2.3	Enfoques clásicos basados en frames	12
2.3.1	Clasificación por frames y piano-roll	12
2.3.2	Limitaciones	12
2.4	Modelos de secuencia a secuencia y generación de eventos	13
2.4.1	Audio a secuencia de tokens	13
2.4.2	El Transformer en el paradigma Seq2Seq	13
2.5	La arquitectura Transformer	14
2.5.1	El mecanismo de autoatención	14
2.5.2	Codificación posicional	14

2.5.3	Configuración Encoder-Decoder	15
2.5.4	Aplicación al problema de la AMT	15
2.6	Atención eficiente y dispersa	17
2.6.1	Coste cuadrático de la autoatención.	17
2.6.2	Ventanas deslizantes frente a dispersidad dinámica	17
2.7	Métricas de evaluación en AMT	18
2.7.1	Evaluación a nivel de trama.	18
2.7.2	Evaluación a nivel de nota	19
2.7.3	Herramienta de evaluación: <i>mir_eval</i>	19
2.8	Resumen y posicionamiento del trabajo	19
3	Datos y Representación de la Información.	21
3.1	Conjunto de Datos	21
3.1.1	Descripción del corpus.	21
3.1.2	Partición de los datos	22
3.2	Representación de entrada: Dominio Acústico	23
3.2.1	Preprocesamiento de la señal	23
3.2.2	Parámetros del espectrograma Log-Mel.	24
3.3	Representación de salida: Dominio Simbólico.	24
3.3.1	Diseño del vocabulario (Tokenization).	24
3.3.2	Gramática de eventos y lógica de decodificación.	25
4	Arquitectura Neuronal Propuesta	27
4.1	Visión general del modelo	27
4.2	Codificador con atención dispersa (Sparsifiner)	29
4.2.1	Proyección de entrada y embedding posicional	29
4.2.2	Bloques de atención Sparsifiner	29
4.3	Mecanismo de predicción de máscara de atención	30
4.3.1	Compresión de rango	30
4.3.2	Cálculo de atención aproximada de bajo rango.	31
4.3.3	Generación de la máscara binaria de atención	31
4.3.4	Atención completa con máscara dispersa	31
4.4	Decodificador autorregresivo	32
4.4.1	Embedding de entrada y codificación posicional	32
4.4.2	Estructura de cada capa del decodificador	33
4.4.3	Capa de clasificación y generación	33
4.5	Resumen de la arquitectura y flujo de datos.	34

5 Implementación.....	37
5.1 Entorno de desarrollo	37
5.2 Pipeline de datos	38
5.2.1 Procesamiento de audio	38
5.2.2 Tokenización MIDI.....	39
5.2.3 Carga y empaquetado de datos	39
5.3 Procedimiento de entrenamiento	39
5.3.1 Estrategia de entrenamiento del decodificador	39
5.3.2 Función de pérdida	40
5.3.3 Optimización	40
5.3.4 Técnicas de estabilización	40
5.3.5 Gestión de checkpoints	40
5.4 Diseño experimental	40
5.5 Sistema de inferencia y evaluación	42
5.5.1 Estrategias de decodificación.....	42
5.5.2 Protocolo de evaluación	42
6 Resultados Experimentales	43
6.1 Protocolo de evaluación	43
6.2 Curvas de entrenamiento	44
6.2.1 v1_baseline	44
6.2.2 exp01_reduced_data	46
6.2.3 exp02_short_clips.....	48
6.2.4 Comparativa entre experimentos	49
6.3 Análisis cualitativo	51
6.3.1 Patrón de predicción de notas	51
6.3.2 Brecha entre onset y onset+offset	52
6.3.3 Efecto de la longitud del clip de entrenamiento	52
6.3.4 Varianza entre piezas	53
6.4 Discusión	54
6.4.1 Rendimiento en perspectiva.....	54
6.4.2 Interpretación experimental	54
6.4.3 Limitaciones de la evaluación	55
7 Conclusiones	57
7.1 Resumen del trabajo	57
7.2 Contribuciones.....	59

7.3 Competencias desarrolladas	60
7.4 Limitaciones	61
7.5 Trabajo futuro	62
Referencia bibliográfica.....	64
Declaración de uso de Inteligencia Artificial	65

Índice de figuras

2.1	De la señal de audio al espectrograma. Superior: forma de onda de una grabación de piano. Inferior: espectrograma de magnitudes obtenido mediante la STFT. Las regiones más brillantes indican mayor energía espectral; los ataques de notas se aprecian como transitorios verticales de alta energía, y las notas sostenidas como bandas horizontales.	9
2.2	Espectrograma Log-Mel de una interpretación de piano. En comparación con el espectrograma lineal, la escala Mel distribuye los filtros de forma más densa en las frecuencias graves —donde se concentra la mayor parte de la información armónica del piano— y más dispersa en las agudas, emulando la percepción auditiva humana.	10
2.3	Ejemplo de representación <i>piano-roll</i> : el eje horizontal codifica el tiempo y el eje vertical los 88 semitonos del piano. Cada rectángulo representa una nota activa, con su duración y dinámica codificadas visualmente.	11
2.4	Configuración <i>Encoder-Decoder</i> del Transformer [Vaswani et al., 2017]. Las entradas y salidas (en gris) se procesan fuera de los bloques principales. El codificador produce la <i>Memoria</i> , de la que el decodificador extrae claves (K) y valores (V) mediante atención cruzada en cada una de sus L_{dec} capas. Las cajas punteadas delimitan los bloques que se apilan L veces.	16
3.1	Alineación temporal entre la señal de audio y el <i>piano-roll</i> MIDI en una interpretación del corpus MAESTRO (Frédéric Chopin, <i>Ballade</i> n.º 1 en Sol menor, Op. 23). Panel superior: forma de onda normalizada de la grabación acústica. Panel inferior: <i>piano-roll</i> MIDI correspondiente, con las notas coloreadas según su velocidad de pulsación. La línea vertical naranja señala un ataque concreto, evidenciando la precisión de alineación inferior a 3 ms que caracteriza al <i>dataset</i> MAESTRO.	22
6.1	Evolución de la pérdida de entrenamiento de <code>v1_baseline</code> entre las épocas 52 y 100 (corpus completo, <code>MAX_AUDIO_FRAMES</code> = 4 096).	44
6.2	Métricas F1 de validación de <code>v1_baseline</code> . El mejor <i>checkpoint</i> corresponde a la época 98.	45

6.3	Evolución de la pérdida de entrenamiento de exp01_reduced_data (150 muestras de entrenamiento).	46
6.4	Métricas F1 de validación de exp01_reduced_data. Se empleó el <i>checkpoint</i> de la última época disponible (514) al no disponer de ranking de pérdida de validación.	47
6.5	Evolución de la pérdida de entrenamiento de exp02_short_clips (corpus completo, MAX_AUDIO_FRAMES = 2 048).	48
6.6	Métricas F1 de validación de exp02_short_clips. El mejor <i>checkpoint</i> corresponde a la época 57.	49
6.7	Comparativa de las curvas de pérdida de entrenamiento de los tres experimentos.	50
6.8	Comparativa de las métricas F1 de validación (<i>onset</i> , <i>onset+offset</i> y <i>onset+velocity</i>) de los tres experimentos.	50

Índice de tablas

3.1	Estadísticas de las particiones (<i>splits</i>) del conjunto de datos MAESTRO [Hawthorne et al., 2019]. *Una misma obra musical puede estar interpretada por varios pianistas distintos dentro del corpus; el recuento de composiciones únicas es por tanto una estimación basada en los metadatos oficiales del <i>dataset</i>	23
3.2	Parámetros de configuración para el cálculo del espectrograma Log-Mel.	25
4.1	Hiperparámetros del modelo Magdalena v1.0 en la configuración experimental. †El código soporta una configuración de mayor capacidad ($d = 512$, $H = 8$, $L_{enc} = L_{dec} = 6$), equivalente al Transformer original [Vaswani et al., 2017], descartada por las limitaciones de memoria de la GPU disponible (Google Colab, T4 de 15 GB de VRAM). La configuración reducida mantiene un <i>batch</i> efectivo de 32 secuencias en todos los experimentos, alcanzado mediante distintas combinaciones de tamaño de <i>batch</i> y acumulación de gradientes según las restricciones de memoria de cada configuración (véase Capítulo 5).	28
5.1	Stack tecnológico del proyecto.	38
5.2	Parámetros variables por experimento. El lote efectivo se mantiene constante en 32 en todas las configuraciones mediante distintas combinaciones de <code>BATCH_SIZE</code> y acumulación de gradientes.	41
5.3	Criterios de acierto para cada variante del F1-score de nivel de nota, implementados con <code>mir_eval</code>	42
6.1	Métricas de evaluación sobre el conjunto de prueba (30 piezas, semilla 42). Los valores se calculan pieza a pieza y se agregan mediante media y desviación típica.	51

1. Introducción General

El presente capítulo establece el marco fundacional y la justificación académica de este Trabajo Fin de Grado. Su propósito es proporcionar una visión global del problema abordado, delimitando el alcance del proyecto y definiendo las metas a alcanzar. A lo largo de las siguientes secciones, se contextualiza la transcripción automática de música, se expone la motivación técnica detrás de la optimización de modelos de aprendizaje profundo y se detallan tanto los objetivos principales como las competencias adquiridas. Finalmente, se enumeran las contribuciones científicas y prácticas derivadas de este trabajo y se presenta la estructura organizativa del resto del documento.

1.1. Contextualización del área de estudio

Durante la última década, la Inteligencia Artificial y el Procesamiento Digital de Señales han transformado profundamente la forma en que los sistemas computacionales analizan e interpretan la información multimedia. En particular, las arquitecturas de aprendizaje profundo (*Deep Learning*) han supuesto un cambio de paradigma en el análisis de audio, superando las limitaciones de los métodos heurísticos tradicionales y permitiendo extraer patrones complejos a partir de grandes volúmenes de datos acústicos que, hasta entonces, resultaban difíciles de procesar e interpretar de forma automática.

El análisis computacional de la música no solo persigue una comprensión más profunda de las estructuras sonoras, sino que habilita aplicaciones de alto impacto práctico. Entre ellas destacan la creación de herramientas interactivas para la educación musical [Benetos et al., 2019], el desarrollo de sistemas de acompañamiento automático [Cont, 2010], el análisis musicológico a gran escala [Müller, 2021] y la preservación del patrimonio cultural mediante la digitalización de grabaciones históricas [Benetos et al., 2019].

La transcripción automática de música (del inglés, *Automatic Music Transcription*, en adelante AMT) aborda el desafío de convertir una señal de audio sin procesar en una representación simbólica estructurada, tal como una partitura o un archivo MIDI. En el caso específico del piano, la complejidad de este proceso se incrementa debido a su inherente naturaleza polifónica. La superposición de notas simultáneas, la variabilidad dinámica (es decir, la intensidad de interpretación), el uso del pedal y la presencia de resonancias armónicas dificultan en gran medida la separación y temporización de los eventos musicales, haciendo de la AMT de piano uno de los problemas abiertos más activos en MIR (del inglés, *Music Information Retrieval*).

El presente Trabajo Fin de Grado propone el diseño y desarrollo de un sistema de AMT especializado en piano, capaz de transformar grabaciones de audio en secuencias de eventos MIDI. A diferencia de los enfoques clásicos, que analizan el audio instante a instante de forma independiente, este proyecto aborda la transcripción como un proceso de traducción en el que el sistema considera la pieza musical en su conjunto, capturando dependencias entre eventos distantes para generar una representación simbólica que codifica de manera integral el tono, la duración, la dinámica y los silencios.

1.2. Motivación y planteamiento del problema

La transcripción de música, en el caso del piano especialmente, presenta desafíos acústicos y musicales únicos que exigen un análisis computacional a largo plazo. Elementos expresivos fundamentales, como el uso del pedal de resonancia (*sustain*), provocan que la estela acústica de las notas se prolongue significativamente en el tiempo, superponiéndose de manera compleja con eventos musicales posteriores. Para modelar correctamente estas dependencias temporales extendidas y desentramar la polifonía, los sistemas automáticos no pueden limitarse a analizar ventanas de audio cortas de forma aislada; requieren comprender un contexto temporal mucho más amplio.

Si bien las arquitecturas previas basadas en Redes Neuronales Convolucionales (CNN) o Recurrentes (RNN) han logrado avances notables en la disciplina MIR, presentan limitaciones inherentes para capturar dependencias a muy largo plazo de manera paralela y eficiente. En este contexto, la creciente disponibilidad de grandes colecciones musicales digitalizadas y el auge de los modelos basados en mecanismos de atención (*transformers*) han impulsado nuevos enfoques en la AMT capaces de integrar dicho contexto extendido.

Sin embargo, los modelos Transformer estándar presentan una limitación práctica importante cuando se aplican al audio continuo: para relacionar cada instante de la pieza con todos los demás, el coste de cálculo y memoria crece de forma cuadrática con la duración de la grabación. En la práctica, esto significa que procesar una pieza de piano de cinco minutos requiere una capacidad de cómputo que resulta prohibitiva incluso con hardware especiali-

zado de alto rendimiento.

Para abordar este problema, este proyecto parte de una observación clave: no todos los instantes de una pieza musical contienen la misma cantidad de información relevante. Los momentos en que se pulsa una tecla, cambia la armonía o el pedal altera el sonido concentran la mayor parte de la información útil para la transcripción; los silencios y los sonidos sostenidos, en cambio, aportan mucho menos. Aprovechando esta asimetría, el sistema aprende a centrar su atención en los instantes más informativos e ignorar los redundantes, reduciendo el coste computacional sin perder precisión en la transcripción.

1.3. Objetivos

El objetivo principal de este Trabajo Fin de Grado es *diseñar, implementar y evaluar un sistema de transcripción automática de música para piano polifónico capaz de convertir señales de audio sin procesar en secuencias MIDI, maximizando la fidelidad temporal y dinámica de la transcripción.*

Para alcanzar esta meta global, se plantean los siguientes objetivos específicos, alineados con las fases de desarrollo del proyecto:

- **Diseño del flujo de preprocesamiento:** Implementar un sistema de extracción de características basado en espectrogramas Log-Mel que capture adecuadamente las propiedades acústicas del piano.
- **Definición de la representación simbólica:** Establecer un vocabulario basado en *tokens* capaz de codificar de manera integral los eventos musicales (activación y desactivación de notas, desplazamientos temporales y dinámica).
- **Desarrollo de la arquitectura neuronal:** Diseñar e implementar un modelo de aprendizaje profundo capaz de generar, paso a paso, la secuencia de eventos MIDI a partir de la representación acústica de la pieza de piano.
- **Optimización computacional:** Investigar y adaptar un mecanismo de atención eficiente que permita procesar grabaciones de larga duración sin sacrificar la capacidad del modelo para capturar relaciones musicales entre eventos distantes en el tiempo.
- **Evaluación y validación:** Establecer un marco de evaluación empírico y riguroso utilizando métricas estándar de la disciplina MIR (como el *F1-score*) para evaluar con precisión la detección de inicios (*onsets*), finales (*offsets*) y velocidad de las notas.

El desarrollo del proyecto evidencia la puesta en práctica de las siguientes competencias de la especialidad en Computación:

- **[CM3]** Capacidad para evaluar la complejidad computacional de un problema, conocer estrategias algorítmicas que puedan conducir a su resolución y recomendar, desarrollar e implementar aquella que garantice el mayor rendimiento de acuerdo con los requisitos establecidos.
- **[CM4]** Capacidad para conocer los fundamentos, paradigmas y técnicas propias de los sistemas inteligentes y analizar, diseñar y construir sistemas, servicios y aplicaciones informáticas que utilicen dichas técnicas en cualquier ámbito de aplicación.
- **[CM5]** Capacidad para adquirir, obtener, formalizar y representar el conocimiento humano en una forma computable para la resolución de problemas mediante un sistema informático en cualquier ámbito de aplicación, particularmente los relacionados con aspectos de computación, percepción y actuación en ambientes o entornos inteligentes.
- **[CM7]** Capacidad para conocer y desarrollar técnicas de aprendizaje computacional y diseñar e implementar aplicaciones y sistemas que las utilicen, incluyendo las dedicadas a extracción automática de información y conocimiento a partir de grandes volúmenes de datos.

1.4. Contribuciones

Entre las principales contribuciones de este Trabajo Fin de Grado al campo de la transcripción automática de música se encuentran:

- **Modelado integral de la AMT como traducción secuencia a secuencia:** Formulación del problema de transcripción de piano como un proceso de traducción directa entre la señal acústica y una secuencia de eventos musicales discretos, codificando de manera integral el tono, la duración, la dinámica y los silencios.
- **Adaptación de atención dispersa al dominio del audio:** Diseño e integración de un mecanismo de atención dispersa, inspirado conceptualmente en el modelo Sparsifiner, adaptado específicamente para procesar representaciones bidimensionales de audio (espectrogramas) con el objetivo de mitigar la complejidad computacional inherente a los Transformers.
- **Implementación y resolución de desafíos de ingeniería:** Desarrollo práctico y optimización del sistema completo utilizando el *framework* PyTorch. Esta contribución incluye la identificación empírica y la resolución de problemas críticos de implementación, tales como la gestión de la coherencia en el vocabulario de *tokens*, la superación de limitaciones estrictas de memoria y la garantía de equidad y consistencia en la evaluación al procesar obras musicales mediante fragmentos (*chunks*).

1.5. Estructura de la memoria

El siguiente trabajo se organiza del siguiente modo:

- **Capítulo 2: Estado del arte** Se establece el marco teórico y se revisa la evolución de los sistemas de AMT, los enfoques basados en aprendizaje profundo y los mecanismos de atención eficiente.
- **Capítulo 3: Datos y Representación de la Información** Describe el conjunto de datos utilizado y detalla el proceso de representación de entrada y salida.
- **Capítulo 4: Arquitectura Neuronal Propuesta** Profundiza en la arquitectura de red propuesta y en la integración matemática del mecanismo de atención dispersa.
- **Capítulo 5: Implementación** Expone los aspectos clave de la implementación práctica y las decisiones de ingeniería adoptadas.
- **Capítulo 6: Resultados Experimentales** Recoge la metodología experimental, presentando y discutiendo los resultados obtenidos.
- **Capítulo 7: Conclusiones** Sintetiza las conclusiones del proyecto y propone posibles líneas de trabajo futuro.

2. Estado del arte

Este capítulo establece los fundamentos teóricos sobre los que se sustenta el presente proyecto y revisa la evolución histórica y tecnológica de la Transcripción Automática de Música (AMT). En primer lugar, se describe cómo se representa el audio para su procesamiento con redes neuronales, detallando la cadena de transformación desde la señal acústica hasta el espectrograma Log-Mel. A continuación, se formaliza el problema de la AMT y las representaciones de entrada y salida habituales en la disciplina. Seguidamente, se analizan los enfoques clásicos de clasificación por tramas y sus limitaciones inherentes, dando paso al estudio del paradigma de traducción de secuencia a secuencia y la generación de eventos musicales discretos. Se profundiza después en la arquitectura Transformer, su mecanismo de atención y su configuración Encoder-Decoder. A continuación se revisan los mecanismos de atención eficiente que permiten aplicar esta arquitectura a señales de audio de larga duración, y se establece el marco de evaluación estándar empleado en la disciplina. Finalmente, se sitúa la propuesta de este trabajo dentro del contexto científico actual.

2.1. Representación del audio para el aprendizaje profundo

Las redes neuronales no operan directamente sobre el sonido tal y como lo percibimos: necesitan una representación numérica compacta que capture la información musicalmente relevante de la señal acústica. Esta sección describe, de forma progresiva, el proceso de transformación de una grabación de audio en la representación bidimensional que sirve de entrada al modelo desarrollado en este proyecto.

2.1.1. La señal de audio digital

El sonido es, en su origen, una variación continua de la presión del aire a lo largo del tiempo. Para procesarlo computacionalmente, esta variación analógica se convierte en una señal discreta mediante un proceso de muestreo: el valor de la onda sonora se registra a intervalos regulares, con una frecuencia denominada *frecuencia de muestreo* (*sampling rate*). Según el teorema de Nyquist-Shannon [Shannon, 1949], para capturar fielmente todos

los componentes frecuenciales de una señal es necesario muestrear al menos al doble de la frecuencia máxima de interés.

El resultado es una secuencia de valores numéricos que representa la amplitud de la onda en cada instante de tiempo. Aunque fiel a la señal original, esta representación plantea un problema práctico para el aprendizaje profundo: una grabación de un minuto, muestreada a una frecuencia estándar, puede contener cientos de miles de valores. Procesar directamente esta secuencia resulta computacionalmente ineficiente y dificulta la extracción de patrones musicales significativos, como alturas de notas o estructuras rítmicas.

2.1.2. Análisis tiempo-frecuencia: la STFT

Para extraer información musicalmente relevante, se recurre al análisis en el dominio tiempo-frecuencia. La herramienta fundamental es la *Transformada de Fourier a Corto Plazo* (*Short-Time Fourier Transform*, STFT), que permite estudiar cómo evoluciona el contenido frecuencial de la señal a lo largo del tiempo [Müller, 2021].

La STFT divide la señal en ventanas temporales de duración fija y solapadas entre sí. Sobre cada ventana se aplica la Transformada de Fourier, que descompone ese fragmento en sus componentes frecuenciales. Para minimizar artefactos en los bordes de cada ventana, se aplica previamente una función de enventanado (típicamente una ventana de Hann) [Harris, 1978]. El resultado es el *espectrograma* de magnitudes: una representación bidimensional en la que el eje horizontal corresponde al tiempo, el eje vertical a la frecuencia, y el color en cada punto indica la energía presente en esa frecuencia en ese instante, tal y como podemos ver en la siguiente figura 2.1.

El espectrograma proporciona una representación visual e intuitiva del contenido musical: los ataques de notas aparecen como transitorios de alta energía, las notas sostenidas como bandas horizontales, y los acordes polifónicos como la superposición de múltiples bandas frecuenciales simultáneas.

2.1.3. El espectrograma Mel logarítmico

El espectrograma lineal captura toda la información frecuencial de la señal, pero no refleja la forma en que el ser humano percibe el sonido. El sistema auditivo humano distingue con mayor precisión entre frecuencias graves que entre frecuencias agudas de diferencia equivalente. Para modelar esta percepción no lineal se introduce la *escala Mel* [Müller, 2021]: una transformación que comprime las frecuencias agudas y expande las graves, de modo que distancias iguales en la escala Mel corresponden a intervalos percibidos como igualmente separados por el oído humano.

En la práctica, esta transformación se implementa en dos pasos sucesivos. Primero, un banco de filtros triangulares distribuidos según la escala Mel agrupa la energía del espectrograma lineal en bandas frecuenciales perceptualmente uniformes, produciendo el espectrograma Mel (*Mel spectrogram*). Segundo, se aplica una compresión logarítmica sobre los valores de energía resultantes, obteniendo el espectrograma Log-Mel (*Log-Mel spectrogram*): esta compresión reduce el rango dinámico de la señal (que en una grabación de piano puede abarcar varios órdenes de magnitud) y estabiliza la varianza de los datos de entrada,

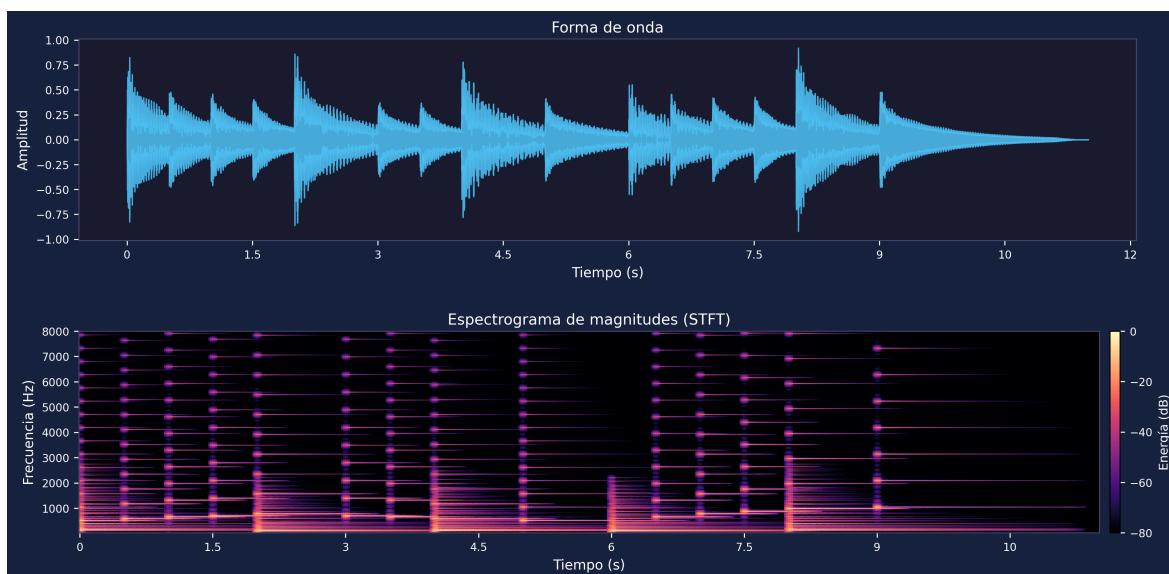


Figura 2.1: De la señal de audio al espectrograma. Superior: forma de onda de una grabación de piano. Inferior: espectrograma de magnitudes obtenido mediante la STFT. Las regiones más brillantes indican mayor energía espectral; los ataques de notas se aprecian como transitorios verticales de alta energía, y las notas sostenidas como bandas horizontales.

facilitando la convergencia del proceso de aprendizaje [Purwins et al., 2019]. La Figura 2.2 ilustra el resultado: las franjas de frecuencias agudas aparecen notablemente más comprimidas que las graves, evidenciando la redistribución perceptual característica de la escala Mel.

El espectrograma Log-Mel se ha consolidado como la representación de entrada estándar para tareas de análisis musical con redes neuronales profundas [Purwins et al., 2019], y es la adoptada en el sistema desarrollado en este proyecto. Los parámetros concretos de su cálculo se detallan en la Sección 3.2, junto con la justificación de cada elección en el contexto de la transcripción de piano.

2.2. Transcripción automática de música (AMT)

2.2.1. Definición del problema

La Transcripción Automática de Música (del inglés, *Automatic Music Transcription*, en adelante AMT) puede definirse formalmente como el proceso computacional que, dada una señal de audio discreta $x \in \mathbb{R}^T$, donde T es el número de muestras de la señal, produce una representación simbólica estructurada $\mathcal{S}$ que describe los eventos musicales subyacentes que la componen [Benetos et al., 2019]. Esta representación puede adoptar múltiples formas, siendo las más habituales el archivo MIDI (*Musical Instrument Digital Interface*) y la

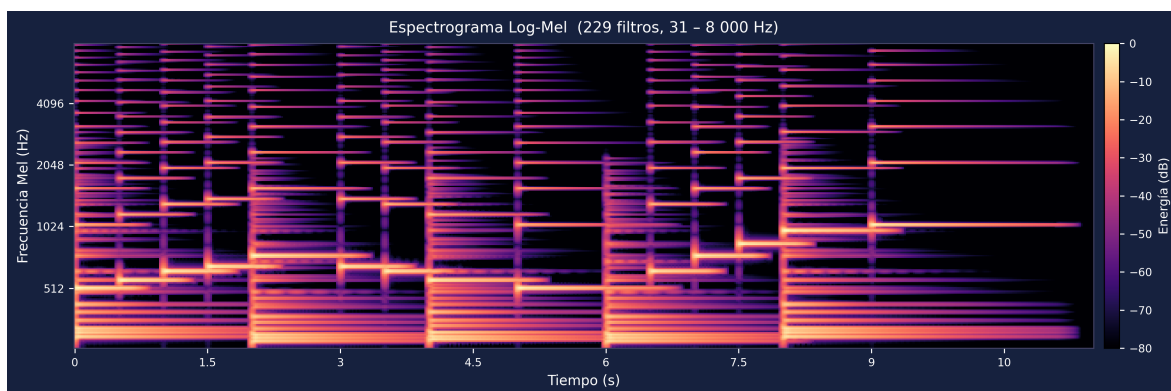


Figura 2.2: Espectrograma Log-Mel de una interpretación de piano. En comparación con el espectrograma lineal, la escala Mel distribuye los filtros de forma más densa en las frecuencias graves —donde se concentra la mayor parte de la información armónica del piano— y más dispersa en las agudas, emulando la percepción auditiva humana.

partitura en notación estándar occidental o su equivalente digital en formato MusicXML.

Desde una perspectiva de aprendizaje automático, la AMT puede encuadrarse dentro de las tareas de reconocimiento estructurado, en las que la salida no es una clase discreta singular, sino una secuencia de longitud variable de eventos interrelacionados. Cada evento musical queda caracterizado por cuatro atributos fundamentales: el tono (*pitch*), que determina la frecuencia fundamental de la nota; el instante de inicio (*onset*), que marca el momento exacto de la pulsación; la duración, que define el intervalo de tiempo hasta la finalización del sonido (*offset*); y la dinámica (*velocity*), que codifica la intensidad de la pulsación [Müller, 2021].

En el caso particular del piano, dominio exclusivo del presente proyecto, la complejidad del problema se ve incrementada significativamente por varios factores acústicos inherentes al instrumento. Su naturaleza intrínsecamente polifónica permite la pulsación simultánea de hasta diez notas de forma independiente, provocando una superposición densa de armónicos en el espectro de frecuencias que dificulta la separación de las fuentes sonoras individuales. Adicionalmente, el uso del pedal de resonancia (*sustain pedal*) extiende artificialmente la duración acústica de las notas, creando interferencias complejas entre eventos musicales sucesivos. La amplia gama dinámica del instrumento, que abarca desde el *pianissimo* hasta el *fortissimo*, introduce variaciones de hasta cuatro órdenes de magnitud en la energía espectral de los ataques. Estos factores hacen que la AMT de piano siga siendo un problema abierto y un área de investigación activa dentro del campo MIR [Benetos et al., 2019].

2.2.2. Representaciones habituales

La elección de las representaciones de los datos de entrada y salida constituye una decisión de diseño fundamental en cualquier sistema de AMT, pues condiciona directamente tanto la arquitectura del modelo como la naturaleza del aprendizaje supervisado.

En el dominio de la entrada acústica, las representaciones más habituales se derivan de la Transformada de Fourier a Corto Plazo (*Short-Time Fourier Transform*, STFT), que permite analizar la evolución temporal del contenido frecuencial de la señal. A partir del espectrograma de magnitudes resultante se han popularizado dos derivaciones fundamentales: el espectrograma en escala Mel (*Mel spectrogram*), que proyecta las frecuencias lineales sobre una escala perceptualmente motivada que emula la respuesta no lineal del sistema auditivo humano, y su versión logarítmica (*Log-Mel spectrogram*), que comprime adicionalmente el rango dinámico y ha demostrado empíricamente ser la representación de referencia para tareas de análisis musical con redes neuronales profundas [Purwins et al., 2019]. Esta última es la adoptada en el presente trabajo, tal y como se detalla en la Sección 3.2.

En lo que respecta a la representación de salida, conviene distinguir entre el enfoque histórico y el moderno, para entender la elección en este proyecto. El formato más extendido históricamente ha sido el *piano-roll*: una representación bidimensional discreta en la que el eje horizontal codifica el tiempo (segmentado en tramas de duración fija) y el eje vertical los 88 semitonos del piano, tal y como se muestra en la figura 2.3, de modo que cada celda activa indica la presencia de una nota en ese instante [Benetos et al., 2019]. Aunque intuitiva, esta representación acopla rígidamente la resolución temporal a la granularidad de la trama, limitando la expresividad de los atributos dinámicos y de duración. Los enfoques más recientes han evolucionado hacia representaciones basadas en eventos discretos: la salida es una secuencia de unidades simbólicas denominadas *tokens*, cada una de las cuales codifica un evento musical específico: la activación de una nota (*Note-On*), su desactivación (*Note-Off*) o un desplazamiento temporal (*Time-Shift*), eliminando así la dependencia de la trama fija [Oore et al., 2020]. Es esta segunda representación la adoptada en el sistema desarrollado en este proyecto, cuyo vocabulario se describe en detalle en la Sección 3.3.

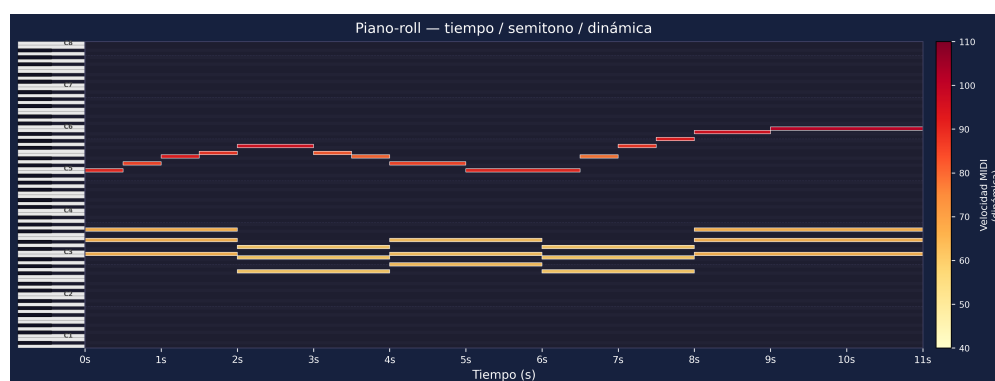


Figura 2.3: Ejemplo de representación *piano-roll*: el eje horizontal codifica el tiempo y el eje vertical los 88 semitonos del piano. Cada rectángulo representa una nota activa, con su duración y dinámica codificadas visualmente.

2.3. Enfoques clásicos basados en *frames*

2.3.1. Clasificación por *frames* y *piano-roll*

Durante la primera generación de sistemas de AMT basados en aprendizaje automático, el paradigma dominante fue la clasificación independiente por tramas (*frame-level classification*). En este enfoque, el modelo procesa una ventana de audio de duración fija (típicamente entre 20 y 100 milisegundos) y produce, para cada trama, un vector binario de 88 dimensiones que indica el estado de activación de cada tecla del piano en ese instante concreto [Benetos et al., 2019]. La concatenación temporal de estos vectores genera la representación *piano-roll*, que posteriormente es procesada mediante heurísticas de post-procesado para agrupar activaciones consecutivas y extraer las notas completas con sus atributos de duración y velocidad.

Las arquitecturas empleadas en esta etapa fueron esencialmente Redes Neuronales Convolucionales (*Convolutional Neural Networks*, CNN), que demostraron ser altamente eficaces para la extracción de características locales en el espectrograma, tales como la detección de ataques transitorios y la identificación de estructuras armónicas periódicas. Trabajos seminales como el de Sigtia et al. [Sigtia et al., 2016] combinaron estas CNN con Redes Neuronales Recurrentes (*Recurrent Neural Networks*, RNN) para incorporar dependencias temporales a corto plazo, sentando las bases de la arquitectura híbrida CNN-RNN que dominó la disciplina durante varios años. En esta misma línea, Kong et al. [Kong et al., 2021] demostraron que modelos de clasificación por frames entrenados con pérdidas específicamente diseñadas para la detección de *onsets* podían alcanzar rendimientos competitivos sobre el *dataset* MAESTRO, obteniendo rendimientos de predicción superiores al 96% bajo condiciones de evaluación estándar.

2.3.2. Limitaciones

A pesar de los resultados destacables que los enfoques basados en *frames* han logrado, presentan limitaciones estructurales que restringen su capacidad de modelado y que motivan directamente la propuesta del presente trabajo.

La limitación más fundamental es la suposición implícita de independencia entre tramas. Al clasificar cada ventana temporal de forma aislada, estos modelos no disponen de mecanismo alguno para razonar sobre la estructura musical global de una pieza: no pueden capturar dependencias armónicas a largo plazo, ni modelar correctamente el arco expresivo de una frase musical, ni anticipar la resolución de una disonancia. Esta miopía temporal limita especialmente la precisión en la detección de *offsets*, ya que la finalización de una nota depende frecuentemente de su contexto melódico y de la interacción con el pedal [Benetos et al., 2019].

Una segunda limitación inherente es la dificultad para modelar de forma conjunta todos los atributos de una nota. La formulación como clasificación binaria en cada trama obliga a tratar el *pitch*, la duración y la velocidad como variables independientes, requiriendo etapas de post-procesado para obtener los eventos completos. Este desacoplamiento introduce errores de propagación que se acumulan a lo largo de la pieza, especialmente en pasajes

con alta densidad polifónica. Finalmente, la dependencia de una resolución temporal fija de la trama impone un compromiso entre resolución temporal y coste computacional que no puede ajustarse dinámicamente en función de la densidad musical local.

2.4. Modelos de secuencia a secuencia y generación de eventos

2.4.1. Audio a secuencia de *tokens*

Para superar las limitaciones de los enfoques basados en *frames*, la comunidad investigadora en MIR ha explorado de forma creciente la reformulación de la AMT como un problema de traducción de secuencia a secuencia. En este paradigma, el sistema aprende un mapeo directo desde una representación continua de la señal acústica hacia una secuencia discreta de *tokens* que codifican los eventos musicales, de manera análoga a la traducción automática en el procesamiento del lenguaje natural (*Natural Language Processing*, NLP) [Oore et al., 2020].

El diseño del vocabulario de *tokens* es un componente central de este enfoque. La representación de eventos propuesta originalmente por Oore et al. [Oore et al., 2020] codifica la música de piano mediante cuatro tipos de eventos:

- $NOTE_ON(p)$: activa la nota de tono p .
- $NOTE_OFF(p)$: desactiva la nota de tono p .
- $TIME_SHIFT(t)$: avanza el cursor temporal t unidades de tiempo.
- $SET_VELOCITY(v)$: establece la intensidad de la nota.

Esta gramática de eventos permite representar de forma compacta y sin pérdida de información cualquier pieza de piano con polifonía arbitraria, eliminando la dependencia de la trama fija y codificando de manera integral los cuatro atributos musicales fundamentales en una única secuencia lineal. El vocabulario implementado en el presente trabajo, descrito en la Sección 3.3, sigue esta misma filosofía con adaptaciones específicas en la cuantización de la velocidad y la resolución temporal.

2.4.2. El Transformer en el paradigma Seq2Seq

La arquitectura Transformer [Vaswani et al., 2017] ha demostrado ser especialmente adecuada para el modelado de la AMT en el paradigma Seq2Seq, gracias a su capacidad para capturar dependencias a largo plazo mediante el mecanismo de autoatención. La resolución correcta de la armonía, la coherencia rítmica y la continuidad del fraseo requieren que el modelo pueda relacionar eventos separados por varios compases, algo que las RNN no logran eficientemente. La arquitectura, su mecanismo interno y su configuración Encoder-Decoder se describen en detalle en la Sección 2.5.

2.5. La arquitectura Transformer

La arquitectura Transformer [Vaswani et al., 2017] ha supuesto un cambio de paradigma en el procesamiento de secuencias, desplazando a las redes recurrentes como estándar de referencia en múltiples dominios. A diferencia de las RNN, que procesan la secuencia elemento a elemento de forma estrictamente secuencial, el Transformer procesa todos los elementos en paralelo, capturando de forma directa las dependencias entre posiciones lejanas. Esta característica lo convierte en una herramienta especialmente adecuada para el dominio musical, donde la comprensión de una nota puede requerir relacionarla con eventos separados por varios compases.

2.5.1. El mecanismo de autoatención

El componente central del Transformer es el mecanismo de *autoatención* (*self-attention*), que permite a cada elemento de una secuencia determinar dinámicamente qué otros elementos son relevantes para su propia representación [Vaswani et al., 2017].

Intuitivamente, el mecanismo puede entenderse así: para cada elemento, el modelo formula una *consulta* (*query*) que expresa qué información necesita; al mismo tiempo, cada elemento publica una *clave* (*key*) que describe qué información ofrece y un *valor* (*value*) que contiene esa información. La atención que el elemento i presta al elemento j se determina midiendo la similitud entre la consulta de i y la clave de j : cuanto mayor sea la similitud, mayor será la ponderación del valor de j en la representación actualizada de i .

Formalmente, dados los tensores de consultas $\mathbf{Q}$, claves $\mathbf{K}$ y valores $\mathbf{V}$, la atención se calcula como:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \quad (2.1)$$

donde d_k es la dimensión de las claves y el factor $\sqrt{d_k}$ escala los productos internos para estabilizar los gradientes durante el entrenamiento. El resultado es una nueva representación de cada elemento que integra, de forma ponderada, la información de todos los demás elementos de la secuencia.

Para capturar relaciones de distinta naturaleza de forma simultánea, el Transformer emplea la *atención multi-cabeza* (*Multi-Head Attention*): el mecanismo de atención se aplica en paralelo varias veces, con proyecciones lineales independientes para cada cabeza. Las salidas de todas las cabezas se concatenan y se proyectan a la dimensión original, enriqueciendo la capacidad representacional del modelo [Vaswani et al., 2017].

2.5.2. Codificación posicional

A diferencia de las redes recurrentes, el Transformer no posee una noción inherente del orden de los elementos en la secuencia: el mecanismo de atención trata todos los pares de posiciones de forma simétrica. Para dotar al modelo de información sobre la posición de cada elemento, se suma a su representación una *codificación posicional* (*positional encoding*) [Vaswani et al., 2017].

La formulación original utiliza funciones sinusoidales de distinta frecuencia para cada dimensión del espacio de representación, de forma que cada posición recibe un vector único e interpretable por el modelo. Una alternativa es aprender las codificaciones posicionales directamente de los datos, lo que permite adaptar la representación posicional a las particularidades del dominio [Dosovitskiy et al., 2021]. La elección entre ambas estrategias tiene implicaciones sobre la capacidad del modelo para generalizar a secuencias de longitud no vista durante el entrenamiento, aspecto especialmente relevante al procesar piezas musicales de duración variable.

2.5.3. Configuración Encoder-Decoder

Para tareas de traducción de secuencias en las que entrada y salida son de distinta naturaleza y longitud, el Transformer adopta una configuración *Encoder-Decoder* [Vaswani et al., 2017], ilustrada en la Figura 2.4.

El *codificador* (*encoder*) recibe la secuencia de entrada completa y la procesa mediante capas sucesivas de autoatención y redes de propagación hacia adelante (*feed-forward networks*), produciendo una secuencia de representaciones contextualizadas que capturan las relaciones globales entre todos los elementos de la entrada.

El *decodificador* (*decoder*) genera la secuencia de salida de forma *autorregresiva*: produce un elemento en cada paso, condicionado tanto a las representaciones del codificador como a todos los elementos ya generados. Para acceder a la información del codificador, cada capa del decodificador incorpora un mecanismo de *atención cruzada* (*cross-attention*), donde las consultas provienen del decodificador y las claves y valores del codificador. Para garantizar que la predicción de cada elemento dependa únicamente de los anteriores, la autoatención del decodificador aplica una *máscara causal* que impide el acceso a posiciones futuras de la secuencia.

2.5.4. Aplicación al problema de la AMT

Esta configuración resulta especialmente adecuada para la transcripción automática de música. El codificador procesa la representación acústica de la pieza (el espectrograma Log-Mel descrito en la Sección 2.1) y el decodificador genera la secuencia de eventos musicales simbólicos que la describe. La generación autorregresiva garantiza la coherencia de la secuencia de salida: cada evento MIDI predicho puede condicionar el siguiente, de manera análoga a la traducción automática en el procesamiento del lenguaje natural.

La ventaja fundamental del Transformer frente a los enfoques clásicos es su capacidad para modelar dependencias a largo plazo [Vaswani et al., 2017]. En el dominio musical, esto resulta crítico: la correcta interpretación de un acorde puede depender del contexto armónico establecido varios compases antes, y la resonancia producida por el pedal de sustain puede solapar notas separadas por varios segundos. El mecanismo de autoatención accede directamente a cualquier posición de la secuencia de entrada sin la degradación de información que afecta a las redes recurrentes en dependencias muy extendidas.

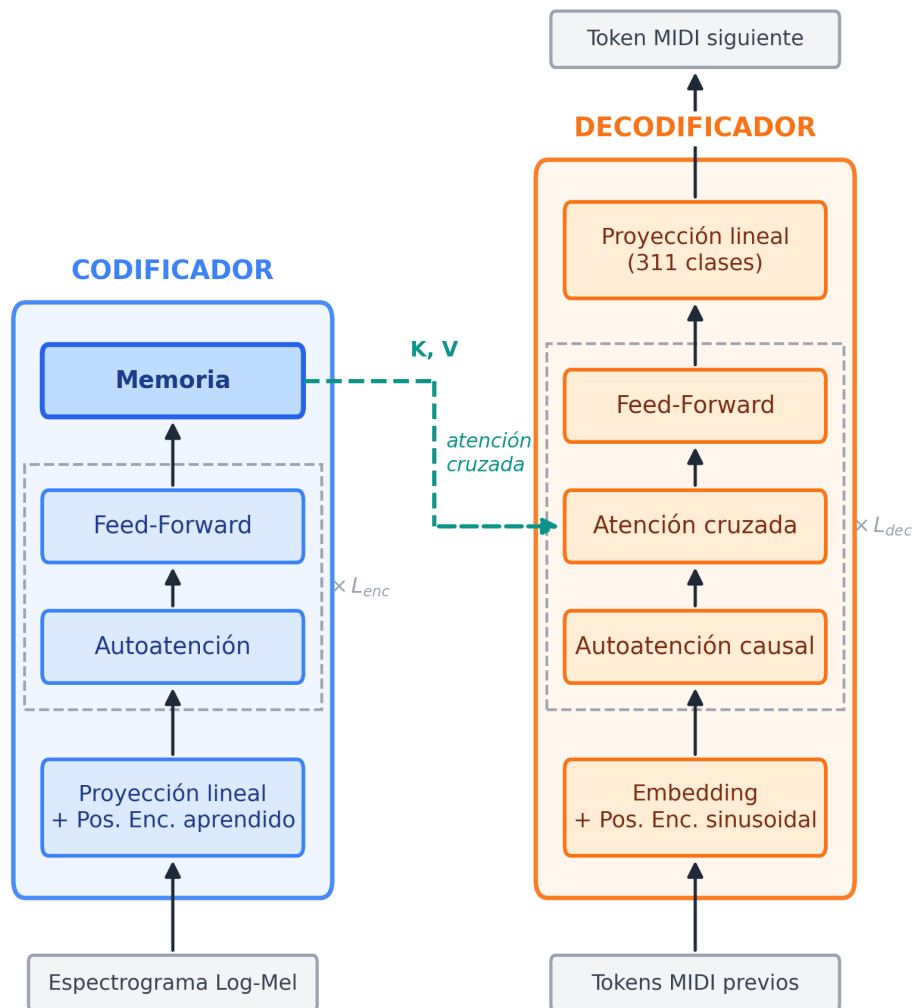


Figura 2.4: Configuración *Encoder-Decoder* del Transformer [Vaswani et al., 2017]. Las entradas y salidas (en gris) se procesan fuera de los bloques principales. El codificador produce la *Memoria*, de la que el decodificador extrae claves (**K**) y valores (**V**) mediante atención cruzada en cada una de sus L_{dec} capas. Las cajas punteadas delimitan los bloques que se apilan L veces.

2.6. Atención eficiente y dispersa

2.6.1. Coste cuadrático de la autoatención

A pesar de sus ventajas en el modelado de dependencias globales, la arquitectura Transformer estándar presenta una limitación computacional crítica cuando se aplica al dominio del audio continuo. El cálculo de la autoatención sobre una secuencia de N tokens requiere construir y operar con una matriz de atención de dimensiones $N \times N$, lo que conlleva una complejidad temporal y espacial de $\mathcal{O}(N^2)$ [Vaswani et al., 2017]. En el dominio del procesamiento del lenguaje natural, donde N raramente supera los miles de tokens, este coste resulta asumible. Sin embargo, en el dominio del audio musical, la situación es radicalmente diferente.

Para ilustrar la magnitud del problema en el contexto concreto de este proyecto: con los parámetros de espectrograma definidos en la Sección 3.2, frecuencia de muestreo $f_s = 16000$ Hz y un *hop size* de 320 muestras (el desplazamiento entre ventanas de análisis consecutivas), cada segundo de audio genera $16000/320 = 50$ tramas espectrales. Una pieza musical de cinco minutos de duración, representativa del corpus MAESTRO, produce por tanto $N = 15000$ tramas. Aplicar autoatención estándar sobre esta secuencia requeriría almacenar una matriz de $15000 \times 15000 = 225 \times 10^6$ valores de atención, con un consumo de memoria que, para precisión de coma flotante de 32 bits, asciende a aproximadamente 900 MB únicamente para la capa de atención. Dado que los modelos profundos apilan múltiples capas de atención y que el entrenamiento requiere almacenar los gradientes intermedios, este coste resulta prohibitivo incluso para hardware de cómputo especializado de alta gama.

2.6.2. Ventanas deslizantes frente a dispersidad dinámica

Para mitigar el cuello de botella cuadrático, se han propuesto en la literatura diversas estrategias de atención eficiente que reducen el número de pares de posiciones que se atienden mutuamente, restringiendo la conectividad de la matriz de atención de forma controlada [Tay et al., 2022].

Un primer grupo de métodos emplea patrones de conectividad fija definidos a priori, independientemente del contenido de la secuencia. El mecanismo de atención por ventana deslizante (*Sliding Window Attention*) es el representante más destacado de esta familia: cada posición i únicamente atiende a las posiciones comprendidas en un intervalo $[i - w, i + w]$, donde w es el tamaño de ventana, reduciendo la complejidad a $\mathcal{O}(N \cdot w)$ [Beltagy et al., 2020]. Este enfoque es computacionalmente eficiente y estructuralmente sencillo, y resulta especialmente apropiado en dominios en los que las dependencias relevantes son predominantemente locales. En el dominio del audio musical, la coherencia acústica a muy corto plazo (como la evolución de un armónico o el transitorio de un ataque) se captura eficazmente mediante ventanas locales. Sin embargo, la atención por ventana deslizante sacrifica por diseño la capacidad de modelar dependencias globales: un acorde en el compás diez no puede influir directamente en la predicción del compás uno si ambos quedan fuera del radio de la ventana.

Un segundo grupo de métodos, conceptualmente más ambicioso, adopta un enfoque de dispersidad dinámica o aprendida (*dynamic sparse attention*): en lugar de definir a priori qué posiciones se atienden entre sí, el modelo aprende a seleccionar, en función del contenido de la secuencia, las conexiones más relevantes para cada instancia concreta [Child et al., 2019]. El modelo *Sparse Transformer* propuesto por Child et al. [Child et al., 2019] demostró que restringir la autoatención a un subconjunto aprendido de conexiones, combinando patrones locales y globales de forma estocástica, permitía procesar secuencias de hasta 12288 *tokens* manteniendo una calidad generativa comparable a la atención completa. En esta misma línea de investigación, el modelo *BigBird* de Zaheer et al. [Zaheer et al., 2020] formalizó teóricamente las garantías de expresividad de los Transformers dispersos, demostrando que la combinación de atención local por ventana, atención global sobre un conjunto reducido de *tokens* de resumen y atención aleatoria es suficiente para aproximar cualquier función computable por un Transformer denso.

La arquitectura Sparsifiner [Wei et al., 2023], que sirve de inspiración directa para el mecanismo de atención implementado en este proyecto, extiende este paradigma al dominio de la visión por computador, interpretando los *tokens* de entrada como parches de una imagen bidimensional y aprendiendo a construir de forma dinámica e instancia-dependiente un grafo de conectividad dispersa sobre dichos parches. Esta perspectiva resulta especialmente pertinente para el dominio del espectrograma de audio, que puede interpretarse conceptualmente como una imagen bidimensional en el plano tiempo-frecuencia: las regiones espectralmente activas, como ataques de notas, transitorios o armónicos prominentes, concentran la mayor densidad de información relevante para la transcripción, mientras que las regiones de silencio o contenido acústico estacionario presentan información redundante y de baja varianza. Frente a la *Sliding Window Attention*, que restringe la conectividad de forma fija e independiente del contenido, Sparsifiner selecciona dinámicamente qué dependencias son relevantes para cada instancia concreta, lo que lo convierte en la estrategia de atención eficiente adoptada en el codificador del presente sistema. Su adaptación al dominio del espectrograma musical se describe en detalle en el Capítulo 4.

2.7. Métricas de evaluación en AMT

La evaluación objetiva de un sistema de AMT requiere métricas que permitan comparar la secuencia de eventos predicha con la verdad base de forma reproducible y comparable con el estado del arte. En este campo coexisten dos paradigmas de evaluación con objetivos complementarios [Benetos et al., 2019].

2.7.1. Evaluación a nivel de trama

El enfoque más sencillo compara el estado de activación de cada nota en cada instante de tiempo, evaluando si el conjunto de notas activas en una trama predicha coincide con el de la trama de referencia. Para cada trama, se calcula el número de verdaderos positivos (notas activas en ambas secuencias), falsos positivos (activas solo en la predicción) y falsos negativos (activas solo en la referencia). A partir de estos valores se derivan la *precisión* P ,

la exhaustividad R y el $F1$ -score:

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F1 = \frac{2 \cdot P \cdot R}{P + R} \quad (2.2)$$

Aunque computacionalmente sencilla, la evaluación por tramas presenta una limitación fundamental: al no modelar explícitamente los eventos de inicio y fin de cada nota, sobrevalora sistemas que predicen correctamente la presencia de una nota pero con duración incorrecta.

2.7.2. Evaluación a nivel de nota

Para una evaluación más rigurosa, el paradigma estándar en la literatura AMT compara las notas como eventos completos, caracterizados por su tono, instante de inicio (*onset*) e instante de fin (*offset*) [Benetos et al., 2019]. Una nota predicha se considera un verdadero positivo si satisface simultáneamente dos condiciones: el tono coincide exactamente con el de la nota de referencia, y su *onset* se encuentra dentro de una ventana de tolerancia de ± 50 ms respecto al *onset* de referencia. Para la evaluación con *offset*, se exige adicionalmente que el *offset* predicho se encuentre dentro de ± 50 ms o del 20 % de la duración de la nota de referencia, el que sea mayor [Müller, 2021].

Esta familia de métricas se desglosa en tres variantes complementarias: evaluación de *onset* únicamente, que mide la capacidad del modelo para detectar el inicio de cada nota; evaluación de *onset* con *offset*, que añade la exigencia de transcribir correctamente la duración; y evaluación de *onset* con velocidad, que valora conjuntamente la detección del ataque y la fidelidad dinámica de la interpretación. Esta combinación permite diagnosticar de forma independiente los distintos aspectos de la transcripción sin recurrir a una métrica única excesivamente penalizadora para un sistema en fase experimental. El protocolo de evaluación concreto aplicado sobre el conjunto de prueba de MAESTRO se detalla en el Capítulo 6.

2.7.3. Herramienta de evaluación: `mir_eval`

El cálculo de todas estas métricas se realiza mediante la biblioteca `mir_eval`, que proporciona una implementación de referencia transparente y reproducible de los estándares de evaluación en Recuperación de Información Musical. Su uso garantiza que los resultados obtenidos son directamente comparables con los publicados en la literatura, eliminando ambigüedades en la definición de las ventanas de tolerancia y en el proceso de emparejamiento entre notas predichas y de referencia.

2.8. Resumen y posicionamiento del trabajo

La revisión del estado del arte presentada en este capítulo permite identificar con claridad la trayectoria evolutiva de la AMT y situar la propuesta del presente proyecto dentro de ella. Los enfoques clásicos basados en clasificación por *frames* sentaron las bases metodológicas de la disciplina y alcanzaron rendimientos notables en condiciones controladas,

pero su incapacidad estructural para modelar dependencias temporales a largo plazo y su tratamiento desacoplado de los atributos musicales constituyen limitaciones difícilmente superables dentro de ese paradigma.

La irrupción de la arquitectura Transformer y su aplicación al problema de la AMT como traducción de secuencia a secuencia ha abierto una vía de investigación cualitativamente diferente, en la que el modelo aprende de forma integral y conjunta todos los atributos de los eventos musicales, capturando dependencias globales gracias al mecanismo de auto-atención. No obstante, el coste computacional cuadrático $\mathcal{O}(N^2)$ de la atención estándar constituye un obstáculo significativo para su aplicación a señales de audio de larga duración, que es precisamente el caso de las interpretaciones de piano presentes en el corpus MAESTRO.

El presente Trabajo Fin de Grado se posiciona en la intersección de estas dos líneas de investigación. Se formula la AMT de piano polifónico como un problema de traducción de secuencia a secuencia, empleando un vocabulario basado en eventos MIDI de 311 *tokens*. Para abordar el cuello de botella computacional de la autoatención estándar, se adopta como mecanismo central del codificador la atención dispersa instancia-dependiente propuesta por Sparsifiner [Wei et al., 2023]: en lugar de restringir la conectividad mediante una ventana local fija, como haría la *Sliding Window Attention*, el modelo aprende a construir dinámicamente el grafo de atención en función del contenido espectral de cada fragmento de audio, seleccionando exclusivamente las dependencias temporales más informativas para la transcripción. Esta elección permite mantener la capacidad de capturar dependencias a largo plazo, esencial para modelar la armonía y el fraseo musical, mientras se reduce el coste efectivo de atención a $\mathcal{O}(r_{attn} \cdot T^2)$ con $r_{attn} = 0.25$. La implementación de este mecanismo, incluyendo las adaptaciones necesarias para el dominio del audio de longitud variable, se detalla en el Capítulo 4.

3. Datos y Representación de la Información

El presente capítulo detalla el flujo de datos implementado para el entrenamiento y evaluación del sistema de transcripción, además del origen de los mismos. Se describe el conjunto de datos utilizado, justificando su idoneidad para el problema polifónico y, más específicamente, el caso concreto del piano. A continuación, se especifica el preprocesamiento de la señal de audio acústica para generar la representación de entrada y, finalmente, se define el diseño del vocabulario discreto que permite codificar los eventos musicales como secuencias de *tokens*.

3.1. Conjunto de Datos

Para el entrenamiento, validación y evaluación del sistema desarrollado, se ha empleado el conjunto de datos MAESTRO (*MIDI and Audio Edited for Synchronous TRacks and Performances*) [Hawthorne et al., 2019]. La elección de este corpus se fundamenta en su posición como el estándar actual en el estado del arte para tareas de transcripción de piano polifónico, superando ampliamente predecesores como MAPS en términos tanto de volumen como de calidad acústica y precisión de alineamiento.

3.1.1. Descripción del corpus

El conjunto de datos MAESTRO consta aproximadamente de 200 horas de interpretaciones virtuosas de obras de piano clásico. La característica más crítica de este corpus para el modelado secuencia a secuencia es la naturaleza de su captura: las interpretaciones fueron registradas utilizando pianos acústicos equipados con el sistema Yamaha Disklavier. Este mecanismo permite capturar simultáneamente la señal de audio acústica de alta fidelidad y los eventos físicos de las teclas y pedales con una precisión temporal inferior a los 3 milisegundos, tal y como se ilustra en la Figura 3.1.

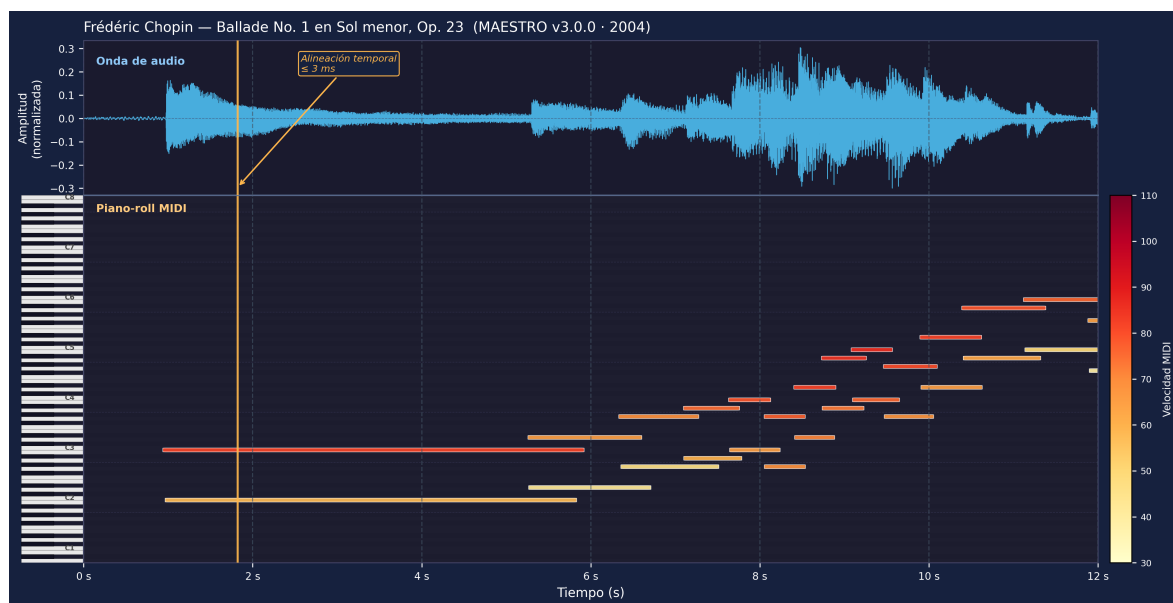


Figura 3.1: Alineación temporal entre la señal de audio y el *piano-roll* MIDI en una interpretación del corpus MAESTRO (Frédéric Chopin, *Ballade* n.º 1 en Sol menor, Op. 23). Panel superior: forma de onda normalizada de la grabación acústica. Panel inferior: *piano-roll* MIDI correspondiente, con las notas coloreadas según su velocidad de pulsación. La línea vertical naranja señala un ataque concreto, evidenciando la precisión de alineación inferior a 3 ms que caracteriza al *dataset* MAESTRO.

Esta alineación temporal precisa entre el dominio continuo (el audio WAV) y el dominio simbólico (los eventos MIDI que lo representan) proporciona la verdad de referencia (*ground truth*) indispensable para supervisar el entrenamiento de la red neuronal. El corpus incluye una enorme variabilidad de condiciones acústicas reales, como el solapamiento de frecuencias, resonancias armónicas complejas producidas por el uso intensivo del pedal de *sustain* y un amplio rango dinámico en la intensidad de pulsación de las teclas (*velocity*), oscilando desde el *pianissimo* hasta el *fortissimo*. Esta riqueza acústica es vital para que el modelo aprenda a desentramar la polifonía densa en lugar de memorizar patrones sintéticos.

3.1.2. Partición de los datos

Para garantizar una metodología de experimentación rigurosa y la capacidad de generalización del modelo, el conjunto de datos se ha dividido en tres subconjuntos mutuamente excluyentes: entrenamiento (*train*), validación (*validation*) y prueba (*test*). Se ha respetado la partición oficial recomendada por los creadores del *dataset*, la cual distribuye los datos en proporciones aproximadas de 80 %, 10 % y 10 %, respectivamente.

Es fundamental destacar que esta partición no se realiza a nivel de fragmento de audio, sino a nivel de composición musical. Esto significa que ninguna obra musical que aparezca en el conjunto de entrenamiento estará presente en los conjuntos de validación o prueba. Esta

restricción de separación estricta (*zero data leakage*) es una decisión de ingeniería crítica que evita que el modelo memorice la estructura de una pieza específica o las particularidades de una grabación concreta. De este modo, las métricas obtenidas durante la evaluación reflejan auténticamente la capacidad del sistema para transcribir piezas musicales nunca antes vistas. Las estadísticas detalladas de cada subconjunto se recogen en la Tabla 3.1 y serán referenciadas en el Capítulo 6 durante el análisis experimental.

<i>Split</i>	<i>Performances</i>	<i>Compositions*</i>	<i>Duration, hours</i>	<i>Size, GB</i>	<i>Notes, millions</i>
<i>Train</i>	954	295	140.1	83.6	5.06
<i>Validation</i>	105	60	15.3	9.1	0.54
<i>Test</i>	125	75	16.9	10.1	0.57
Total	1184	430	172.3	102.8	6.18

Tabla 3.1: Estadísticas de las particiones (*splits*) del conjunto de datos MAESTRO [Hawthorne et al., 2019]. * Una misma obra musical puede estar interpretada por varios pianistas distintos dentro del corpus; el recuento de composiciones únicas es por tanto una estimación basada en los metadatos oficiales del *dataset*

3.2. Representación de entrada: Dominio Acústico

Como se describe en la Sección 2.1, la entrada al modelo es un espectrograma Log-Mel obtenido a partir de la señal de audio en crudo. Esta sección no abarca la justificación conceptual de dicha representación, sino que especifica los parámetros concretos adoptados en este sistema y el flujo de preprocesamiento que los produce.

3.2.1. Preprocesamiento de la señal

El primer paso del flujo de datos consiste en la homogeneización de los archivos de audio de entrada. Todas las grabaciones se convierten de formato estereofónico a monofónico (mezcla de canales) para reducir la carga computacional, dado que la información espacial estéreo no aporta ventajas significativas para la extracción de características armónicas [Müller, 2021]. A continuación, las señales se remuestrean a una frecuencia constante de $f_s = 16000$ Hz. Esta tasa de muestreo es suficiente para capturar el contenido armónico relevante del piano (cuyo rango tonal fundamental llega hasta aproximadamente 4186 Hz), cumpliendo con el teorema de Nyquist-Shannon [Shannon, 1949].

Dado que el corpus MAESTRO contiene interpretaciones de hasta varios minutos de duración, y con el objetivo de mantener la viabilidad computacional del entrenamiento, se aplica una truncación de la secuencia de audio a un máximo de 4 096 tramas espectrales

(equivalente a aproximadamente 82 segundos). Las secuencias más cortas se rellenan con ceros hasta alcanzar la longitud del elemento más largo del lote (*batch*). Los detalles de esta estrategia y su variación entre experimentos se describen en el Capítulo 5.

3.2.2. Parámetros del espectrograma Log-Mel

Siguiendo la representación introducida en la Sección 2.1, el espectrograma Log-Mel se calcula con los parámetros recogidos en la Tabla 3.2. A continuación se justifica cada elección en el contexto específico de la transcripción de piano.

El tamaño de ventana $N_{FFT} = 2048$ muestras (equivalente a 128 ms) proporciona una resolución frecuencial de $f_s/N_{FFT} \approx 7,8$ Hz por banda, suficiente para distinguir notas próximas en el registro grave del instrumento, donde los armónicos están más juntos. El *hop size* de 320 muestras (20 ms por trama) establece la resolución temporal de la representación: a esta granularidad, el modelo puede capturar transitorios de ataque con suficiente precisión para la detección de *onsets* dentro de la ventana de tolerancia de ± 50 ms empleada en la evaluación estándar (véase Sección 2.7). El banco de 229 filtros Mel, delimitado entre $f_{min} = 31$ Hz y $f_{max} = 8000$ Hz, cubre íntegramente el rango tonal del piano (desde A0 a 27,5 Hz hasta C8 a 4186 Hz) incluyendo los armónicos perceptivamente más relevantes. Esta configuración de parámetros sigue la práctica habitual en sistemas de AMT de alto rendimiento evaluados sobre el corpus MAESTRO [Kong et al., 2021, Purwins et al., 2019].

Finalmente, los valores de energía del espectrograma se normalizan al rango $[0, 1]$ mediante dos pasos sucesivos: primero se aplica una conversión logarítmica a decibelios con referencia al máximo de la señal y un rango dinámico de 60 dB, obteniendo valores en el intervalo $[-60, 0]$ dB; a continuación, dichos valores se reescalan linealmente a $[0, 1]$. Esta normalización homogeneiza la distribución de la entrada entre piezas de distinta intensidad media y estabiliza el proceso de entrenamiento.

3.3. Representación de salida: Dominio Simbólico

Para que el modelo Transformer pueda generar eventos musicales mediante un enfoque de traducción de secuencia a secuencia, es necesario transformar los archivos MIDI continuos en una secuencia unidimensional de *tokens* discretos. Para ello, se ha diseñado un vocabulario personalizado inspirado en representaciones estándar de la literatura [Oore et al., 2020].

3.3.1. Diseño del vocabulario (*Tokenization*)

El vocabulario desarrollado consta de un total de 311 *tokens* únicos. Esta dimensionalidad reducida resulta eficiente para la capa de clasificación final de la red neuronal. Los *tokens* se estructuran en familias que cubren integralmente los atributos musicales del piano:

- **Tokens de gestión de secuencia:** El *token* <pad> (índice 0) homogeneiza la longitud de los tensores en los lotes de entrenamiento (*batches*). Los *tokens* <sos> (*Start of*

Parámetro	Valor	Descripción
Frecuencia de muestreo (f_s)	16 000 Hz	Tasa de muestreo de la señal monofónica de entrada.
Tamaño de ventana (N_{FFT})	2048 muestras	Equivalente a 128 ms de resolución temporal por trama.
Tamaño de salto (<i>hop size</i>)	320 muestras	Equivalente a 20 ms de desplazamiento entre ventanas.
Tipo de ventana	<i>Hann</i>	Función aplicada para suavizar los extremos de la trama.
Filtros Mel (N_{mels})	229	Cubre holgadamente las 88 teclas del piano y sus armónicos.
Frecuencia mínima (f_{min})	31 Hz	Límite inferior; próximo a la nota A0 del piano.
Frecuencia máxima (f_{max})	8000 Hz	Límite superior dictado por la frecuencia de Nyquist.

Tabla 3.2: Parámetros de configuración para el cálculo del espectrograma Log-Mel.

Sequence, índice 309) y *<eos>* (*End of Sequence*, índice 310) delimitan las fronteras de la generación autorregresiva.

- **Activación y desactivación de notas (*Note-On / Note-Off*):** Se restringe el rango tonal a las 88 teclas del piano estándar, desde A0 (índice MIDI 21) hasta C8 (índice MIDI 108), lo que se traduce en 88 *tokens* de *Note-On* (índices 1–88) y 88 *tokens* de *Note-Off* (índices 89–176).
- **Desplazamiento temporal (*Time-Shift*):** En lugar de predecir el estado del piano en cada instante físico, el modelo predice saltos en el tiempo con una resolución de $\Delta t = 1,0 \times 10^{-2}$ s. El vocabulario incluye 100 *tokens* de *Time-Shift* (índices 177–276), permitiendo codificar esperas de hasta 1 segundo por *token*.
- **Dinámica (*Velocity*):** La intensidad de pulsación, codificada en MIDI en 128 valores (0–127), se cuantiza linealmente en 32 niveles o *bins* (índices 277–308). Esta reducción mitiga la dispersión de los datos dinámicos sin una pérdida perceptible de expresividad musical.

3.3.2. Gramática de eventos y lógica de decodificación

La transformación de notas MIDI a *tokens* requiere una ordenación cronológica estricta para garantizar la viabilidad del entrenamiento autorregresivo. Los eventos se ordenan por su marca de tiempo y, ante empates temporales, se otorga prioridad absoluta a los eventos *Note-Off* sobre los *Note-On*. Esta regla gramatical previene colisiones lógicas y asegura que

una tecla finalice su vibración antes de volver a ser percutida en el mismo instante.

El proceso inverso (decodificación de *tokens* a archivo MIDI) implementa una máquina de estados para rastrear las notas activas. Esta lógica incluye mecanismos de seguridad robustos: el cierre forzado de una nota si se predice un nuevo *Note-On* sin *Note-Off* previo, y un proceso de limpieza final que apaga todas las notas activas al predecir el *token* <eos>, evitando corrupciones en el archivo MIDI resultante.

4. Arquitectura Neuronal Propuesta

El presente capítulo describe en detalle la arquitectura neuronal diseñada e implementada para el sistema de transcripción automática de piano. Partiendo de los fundamentos teóricos establecidos en el Capítulo 2 y de las representaciones definidas en el Capítulo 3, se describe la configuración global del modelo, la estructura del codificador basado en atención dispersa, el mecanismo de predicción de máscara que constituye el núcleo de la contribución técnica de este trabajo, y el decodificador autorregresivo que genera la secuencia de *tokens* MIDI. Finalmente, se detalla el esquema de codificación posicional adoptado y se justifican las decisiones de diseño más relevantes.

4.1. Visión general del modelo

Siguiendo el paradigma de traducción de secuencia a secuencia descrito en la Sección 2.4, el sistema establece un mapeo directo entre la representación acústica de la pieza y la secuencia simbólica de eventos MIDI. El sistema desarrollado en este proyecto, denominado *Magdalena v1.0*, adopta una arquitectura Encoder-Decoder asimétrica: el codificador y el decodificador difieren tanto en su estructura interna como en el mecanismo de atención que emplean, lo que distingue esta propuesta de las implementaciones estándar del Transformer original [Vaswani et al., 2017].

El flujo de procesamiento del modelo puede describirse formalmente como sigue. Dado un espectrograma Log-Mel de entrada $\mathbf{X} \in \mathbb{R}^{B \times N_{mels} \times T}$, donde B es el tamaño del lote (*batch size*), $N_{mels} = 229$ es el número de bandas frecuenciales y T es la longitud temporal de la secuencia, el codificador produce una representación latente contextualizada $\mathbf{M} \in \mathbb{R}^{B \times T \times d}$, denominada memoria (*memory*), con dimensión de embedding $d = 256$. El decodificador consume esta memoria y genera, de forma autorregresiva, la secuencia de logits $\mathbf{L} \in \mathbb{R}^{B \times S \times |\mathcal{V}|}$, donde S es la longitud de la secuencia objetivo y $|\mathcal{V}| = 311$ es el tamaño del vocabulario definido en la Sección 3.3. La configuración Encoder-Decoder y el mecanismo de autoatención subyacente fueron descritos en detalle en la Sección 2.5.

La Tabla 4.1 recoge los hiperparámetros del modelo tal y como están configurados en los experimentos realizados.

Hiperparámetro	Valor	Descripción
d (embed_dim)	256 [†]	Dimensión del espacio de <i>embedding</i> en todas las capas.
H (nhead)	4 [†]	Número de cabezas de atención (<i>multi-head</i>).
L_{enc} (num_encoder_layers)	4 [†]	Profundidad del codificador (bloques Sparsifiner).
L_{dec} (num_decoder_layers)	4 [†]	Profundidad del decodificador (bloques Transformer estándar).
d_{ff} (dim_feedforward)	2048	Dimensión de la red <i>feed-forward</i> interna ($8 \times d$).
p (dropout)	0.1	Tasa de <i>dropout</i> aplicada en <i>embeddings</i> y atención.
T_{train} (max_audio_frames)	2 048 – 4 096	Longitud máxima de secuencia de audio en entrenamiento (varía por experimento, véase Capítulo 5); capacidad máxima del modelo: 70 000 tramas.
r_{attn} (attn_keep_rate)	0.25	Fracción de conexiones de atención retenidas por Sparsifiner.
r_N (reduce_n_factor)	64	Factor de compresión temporal en el predictor de máscara.
$DropPath$ (drop_path_rate)	0.0	Regularización de profundidad estocástica implementada pero configurada como inactiva en los experimentos actuales (<code>nn.Identity</code>).

Tabla 4.1: Hiperparámetros del modelo Magdalena v1.0 en la configuración experimental. [†]El código soporta una configuración de mayor capacidad ($d = 512$, $H = 8$, $L_{enc} = L_{dec} = 6$), equivalente al Transformer original [Vaswani et al., 2017], descartada por las limitaciones de memoria de la GPU disponible (Google Colab, T4 de 15 GB de VRAM). La configuración reducida mantiene un *batch* efectivo de 32 secuencias en todos los experimentos, alcanzado mediante distintas combinaciones de tamaño de *batch* y acumulación de gradientes según las restricciones de memoria de cada configuración (véase Capítulo 5).

4.2. Codificador con atención dispersa (Sparsifiner)

El codificador del sistema, implementado en la clase `AudioSparsifinerEncoder`, constituye la principal contribución técnica de esta arquitectura. A diferencia del codificador estándar del Transformer, que aplica autoatención densa con complejidad $\mathcal{O}(T^2)$, el codificador propuesto integra el mecanismo Sparsifiner [Wei et al., 2023], que aprende a construir de forma dinámica y dependiente de la instancia un grafo de conectividad dispersa sobre la secuencia de tramas del espectrograma.

4.2.1. Proyección de entrada y embedding posicional

El primer componente del codificador es una capa de proyección lineal $\mathbf{W}_{in} \in \mathbb{R}^{N_{mels} \times d}$ que transforma cada trama frecuencial del espectrograma —un vector de dimensión $N_{mels} = 229$ — en un vector de embedding de dimensión $d = 256$. Formalmente, dado el espectrograma de entrada $\mathbf{X} \in \mathbb{R}^{B \times N_{mels} \times T}$, se permutan las dimensiones frecuencial y temporal para obtener $\mathbf{X}' \in \mathbb{R}^{B \times T \times N_{mels}}$, y la proyección se aplica sobre la última dimensión:

$$\mathbf{E} = \mathbf{X}' \mathbf{W}_{in} + \mathbf{b}_{in} \in \mathbb{R}^{B \times T \times d} \quad (4.1)$$

A la representación resultante se le suma un embedding posicional aprendido $\mathbf{P} \in \mathbb{R}^{1 \times T_{max} \times d}$, inicializado con distribución normal truncada de desviación estándar $\sigma = 0.02$ y recortado dinámicamente a la longitud real de la secuencia en cada paso de inferencia:

$$\mathbf{Z}^{(0)} = \mathbf{E} + \mathbf{P}_{[:,T,:]} \in \mathbb{R}^{B \times T \times d} \quad (4.2)$$

La elección de un embedding posicional aprendido, en lugar del encoding sinusoidal fijo utilizado en el decodificador, está motivada por la naturaleza bidimensional del espectrograma: al interpretar las tramas temporales como *tokens* de una representación matricial, el modelo puede aprender patrones posicionales específicos del dominio acústico que no están bien capturados por funciones periódicas fijas [Dosovitskiy et al., 2021].

4.2.2. Bloques de atención Sparsifiner

El núcleo del codificador está formado por una pila de $L_{enc} = 4$ bloques Sparsifiner idénticos en estructura, implementados en la clase `Block`. Cada bloque sigue la arquitectura Pre-LayerNorm estándar del Transformer [Xiong et al., 2020], con conexiones residuales y una red de propagación hacia adelante (*feed-forward network*, FFN) con función de activación GELU [Hendrycks and Gimpel, 2016]:

$$\mathbf{Z}'^{(l)} = \mathbf{Z}^{(l-1)} + \text{DropPath}(\text{SparseAttn}(\text{LN}(\mathbf{Z}^{(l-1)}), \boldsymbol{\pi}^{(l-1)})) \quad (4.3)$$

$$\mathbf{Z}^{(l)} = \mathbf{Z}'^{(l)} + \text{DropPath}(\text{FFN}(\text{LN}(\mathbf{Z}'^{(l)}))) \quad (4.4)$$

donde $\boldsymbol{\pi}^{(l-1)} \in \{0, 1\}^{B \times (T-1)}$ es la máscara de *tokens* activos propagada desde el bloque anterior, $\text{LN}(\cdot)$ denota normalización de capa (*Layer Normalization*) [Ba et al., 2016], y

DropPath($\cdot$) es la regularización de profundidad estocástica (*Stochastic Depth*) [Huang et al., 2016], que descarta bloques residuales completos de forma aleatoria durante el entrenamiento para prevenir el sobreajuste. En los experimentos actuales esta regularización se mantiene inactiva (`drop_path_rate = 0.0`), equivaliendo a la función identidad; la arquitectura la soporta de forma nativa para experimentos futuros con modelos más profundos. La dimensión oculta de la FFN es $d_{ff} = 2048$, lo que supone una relación $d_{ff} = 8d$ con la configuración experimental ($d = 256$). Esta proporción, mayor que la relación $4d$ convencional del Transformer original [Vaswani et al., 2017], compensa parcialmente la reducción de d manteniendo la capacidad expresiva de la red de propagación hacia adelante.

4.3. Mecanismo de predicción de máscara de atención

El componente diferenciador de la arquitectura es el `MaskPredictor`, un módulo auxiliar de bajo coste computacional que, a partir de las representaciones de consulta (*query*) y clave (*key*) de la capa de atención, predice qué pares de posiciones son suficientemente relevantes para justificar el cómputo de atención completa. Este predictor opera en tres fases: compresión de rango, cálculo de atención aproximada y generación de la máscara binaria final.

4.3.1. Compresión de rango

Para calcular el mapa de relevancia entre posiciones sin incurrir en el coste cuadrático de la atención completa, el `MaskPredictor` proyecta las representaciones de consulta y clave a un espacio de dimensión reducida mediante dos operaciones de compresión independientes y paralelas.

La primera compresión actúa sobre la dimensión de canales: cada cabeza de atención tiene dimensión $d_h = d/H = 64$, y se proyecta a una dimensión reducida $d_c = d_h/r_c = 32$, donde el factor de reducción de canal es $r_c = 2$:

$$\tilde{\mathbf{Q}} = \mathbf{Q} \mathbf{W}_{Q_c} \in \mathbb{R}^{B \times H \times T \times d_c} \quad (4.5)$$

$$\tilde{\mathbf{K}} = \mathbf{K} \mathbf{W}_{K_c} \in \mathbb{R}^{B \times H \times T \times d_c} \quad (4.6)$$

La segunda compresión actúa sobre la dimensión temporal: la matriz de claves reducida se proyecta sobre un conjunto de $k = \lfloor T/r_N \rfloor$ vectores base aprendidos, donde el factor de reducción temporal es $r_N = 64$ (definido en `transformer.py`). Esta proyección se realiza mediante una matriz de parámetros aprendidos $\mathbf{P}_N \in \mathbb{R}^{T_{max} \times k}$, recortada dinámicamente a la longitud real de la secuencia T en cada paso:

$$\hat{\mathbf{K}} = \tilde{\mathbf{K}}^\top \mathbf{P}_{N[:T,:]} \in \mathbb{R}^{B \times H \times d_c \times k} \quad (4.7)$$

El resultado es una representación comprimida de las claves en un espacio latente de $k \ll T$ vectores base, cuyo coste de almacenamiento es $\mathcal{O}(T \cdot k)$ en lugar de $\mathcal{O}(T^2)$.

4.3.2. Cálculo de atención aproximada de bajo rango

A partir de las representaciones comprimidas, el predictor calcula una atención aproximada (*cheap attention*) mediante la siguiente operación:

$$\mathbf{A}_{cheap} = \text{softmax}\left(\tilde{\mathbf{Q}} \cdot \hat{\mathbf{K}} \cdot s\right) \in \mathbb{R}^{B \times H \times T \times k} \quad (4.8)$$

donde $s = H^{-1/2}$ es el factor de escala estándar. Los coeficientes resultantes $\mathbf{A}_{cheap}$ codifican, para cada posición temporal, el grado de afinidad con cada uno de los k vectores base del espacio comprimido. Estos coeficientes se utilizan para reconstruir una aproximación de la matriz de atención completa $N \times N$ mediante la proyección inversa $\mathbf{P}_N^\top$:

$$\hat{\mathbf{A}} = \mathbf{A}_{cheap} \cdot \mathbf{P}_{back}^\top \in \mathbb{R}^{B \times H \times T \times T} \quad (4.9)$$

donde $\mathbf{P}_{back} \in \mathbb{R}^{T_{max} \times k}$ es una segunda matriz de proyección aprendida independiente de $\mathbf{P}_N$ (`share_inout_proj=False`), que permite una mayor flexibilidad en la reconstrucción.

4.3.3. Generación de la máscara binaria de atención

La matriz de atención aproximada $\hat{\mathbf{A}}$ se utiliza para determinar qué conexiones merecen ser calculadas con atención completa. Dado un presupuesto de atención $b = \lceil r_{attn} \cdot T \rceil$, donde $r_{attn} = 0.25$ es la tasa de retención definida en `transformer.py`, se seleccionan para cada posición de consulta los b índices de clave con mayor puntuación aproximada mediante una operación *top-k*:

$$\mathcal{I}^{(i)} = \text{top-}b \hat{A}_{ij} \quad \forall i \in [T] \quad (4.10)$$

A partir de estos índices se construye la máscara binaria de atención $\mathbf{M}_{attn} \in \{0, 1\}^{B \times H \times T \times T}$, donde $M_{attn}^{(i,j)} = 1$ si y solo si $j \in \mathcal{I}^{(i)}$. Esta máscara se desconecta del grafo de gradientes (`.detach_()`) para que la operación *top-k* —no diferenciable— no bloquee la retropropagación. El coste de calcular la máscara es $\mathcal{O}(T \cdot k)$, y el coste de aplicarla en la atención completa se reduce de $\mathcal{O}(T^2)$ a $\mathcal{O}(T \cdot b) = \mathcal{O}(r_{attn} \cdot T^2)$, con $r_{attn} = 0.25$ en este sistema.

Adicionalmente, la implementación introduce un ajuste dinámico del presupuesto (`real_k = min(self.attn_budget, N)`), que garantiza la compatibilidad con secuencias más cortas que la longitud máxima T_{max} sin requerir relleno hasta dicha longitud. Esta es una contribución de ingeniería relevante respecto a la implementación original de Sparsifiner, que asumía una longitud de secuencia fija.

4.3.4. Atención completa con máscara dispersa

Una vez obtenida la máscara $\mathbf{M}_{attn}$, la capa de atención (`Attention`) calcula la matriz de atención estándar sobre las representaciones originales no comprimidas, pero fuerza a $-\infty$ las posiciones no seleccionadas por la máscara antes de aplicar *softmax*:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_h}} + \log \mathbf{M}_{attn}\right) \in \mathbb{R}^{B \times H \times T \times T} \quad (4.11)$$

donde la suma con $\log \mathbf{M}_{attn}$ implementa el enmascaramiento: las posiciones con $M_{attn}^{(i,j)} = 0$ reciben una puntuación de $-\infty$, resultando en un peso de atención efectivo de cero tras el *softmax*. Las filas de la matriz resultante que corresponden a posiciones completamente enmascaradas —situación posible en secuencias con *padding*— son gestionadas mediante `torch.nan_to_num` para garantizar la estabilidad numérica. La salida del módulo de atención es finalmente:

$$\mathbf{Z}_{attn} = \mathbf{A}\mathbf{V}\mathbf{W}_O \in \mathbb{R}^{B \times T \times d} \quad (4.12)$$

donde $\mathbf{V}$ son las representaciones de valor y $\mathbf{W}_O \in \mathbb{R}^{d \times d}$ es la proyección de salida.

4.4. Decodificador autorregresivo

El decodificador del sistema, implementado en la clase `MidiDecoder`, adopta la arquitectura estándar del decodificador Transformer [Vaswani et al., 2017], compuesta por $L_{dec} = 4$ capas apiladas de tipo `nn.TransformerDecoderLayer`. A diferencia del codificador, el decodificador emplea atención densa estándar, decisión justificada por las diferencias fundamentales entre las dos secuencias que procesa: mientras la secuencia de audio puede contener decenas de miles de tramas, la secuencia de *tokens* MIDI generada tiene una longitud típicamente mucho menor, lo que hace manejable el coste cuadrático de la autoatención.

4.4.1. Embedding de entrada y codificación posicional

Cada *token* MIDI de la secuencia objetivo $\mathbf{y} \in \mathbb{Z}^{B \times S}$ se transforma en un vector de embedding de dimensión $d = 256$ mediante una tabla de embeddings aprendida $\mathbf{E}_{tok} \in \mathbb{R}^{|\mathcal{V}| \times d}$, con el índice de *padding* (índice 0) inicializado a cero y excluido de la actualización de gradientes. El embedding resultante se escala por $\sqrt{d}$ para compensar la magnitud del embedding posicional [Vaswani et al., 2017], y se le suma una codificación posicional sinusoidal fija generada mediante la clase `PositionalEncoding`:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad (4.13)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (4.14)$$

La implementación utiliza la formulación logarítmicamente estable $\exp\left(2i \cdot \frac{-\log(10000)}{d}\right)$ para el cálculo del denominador, y el *buffer* posicional se registra como parámetro no entrenable con longitud máxima $max_len = 50\,000$ *tokens*, valor suficientemente holgado para cubrir las secuencias MIDI más largas del corpus MAESTRO. La codificación posicional

sinusoidal se ha elegido para el decodificador —en contraposición al embedding posicional aprendido del codificador— por su capacidad de generalización a longitudes de secuencia no vistas durante el entrenamiento [Vaswani et al., 2017].

4.4.2. Estructura de cada capa del decodificador

Cada capa del decodificador implementa el flujo de procesamiento canónico en tres subcapas:

Autoatención enmascarada (*Masked Self-Attention*). La primera subcapa aplica autoatención multi-cabeza sobre la secuencia de *tokens* ya generados, utilizando una máscara triangular inferior (*causal mask*) para garantizar la propiedad autorregresiva: la predicción del *token* en la posición t solo puede depender de los *tokens* en posiciones $0, 1, \dots, t - 1$. La máscara se genera dinámicamente en cada paso de decodificación mediante:

$$M_{causal}^{(i,j)} = \begin{cases} 0 & \text{si } j \leq i \\ -\infty & \text{si } j > i \end{cases} \quad (4.15)$$

Atención cruzada (*Cross-Attention*). La segunda subcapa aplica atención cruzada entre la secuencia de *tokens* del decodificador (actuando como consultas) y la memoria $\mathbf{M}$ producida por el codificador (actuando como claves y valores). Este mecanismo es el que permite al decodificador acceder al contenido acústico de la pieza en cada paso de generación, condicionando la predicción del siguiente *token* MIDI a la representación latente completa del espectrograma de entrada.

Red *feed-forward*. La tercera subcapa es una red de propagación hacia adelante de dos capas lineales con función de activación ReLU y dimensión oculta $d_{ff} = 2048$, seguida de *dropout* con $p = 0.1$.

Todas las subcapas implementan conexiones residuales y normalización de capa (*Post-LayerNorm*), siguiendo la implementación estándar de PyTorch (`nn.TransformerDecoderLayer` con `batch_first=True`).

4.4.3. Capa de clasificación y generación

La salida de la última capa del decodificador, $\mathbf{H} \in \mathbb{R}^{B \times S \times d}$, se proyecta a logits sobre el vocabulario mediante una capa lineal de salida:

$$\mathbf{L} = \mathbf{H} \mathbf{W}_{out} + \mathbf{b}_{out} \in \mathbb{R}^{B \times S \times |\mathcal{V}|} \quad (4.16)$$

donde $\mathbf{W}_{out} \in \mathbb{R}^{d \times |\mathcal{V}|}$ con $|\mathcal{V}| = 311$ (véase Sección 3.3). Durante el entrenamiento, estos logits se utilizan directamente para calcular la función de pérdida de entropía cruzada con la secuencia objetivo. Durante el entrenamiento se emplea *teacher forcing* estricto: el decodificador recibe como entrada la secuencia de *tokens* objetivo desplazada una posición

(`batch_midi[:, :-1]`), lo que acelera la convergencia pero introduce una discrepancia entre las condiciones de entrenamiento e inferencia (*exposure bias*).

Durante la inferencia el sistema soporta dos estrategias de decodificación. La primera es la decodificación voraz (*greedy decoding*): en cada paso t se selecciona el *token* con mayor logit y se incorpora a la secuencia de entrada para el paso siguiente. La segunda es el muestreo con temperatura (*temperature sampling*), con $temperature = 0.8$, que introduce estocasticidad en la selección mediante `torch.multinomial`, favoreciendo la diversidad de las transcripciones generadas. En ambos casos el proceso se detiene al predecir el *token* `<eos>` (índice 310) o al alcanzar la longitud máxima de generación.

4.5. Resumen de la arquitectura y flujo de datos

El flujo de datos completo a través del modelo fue introducido esquemáticamente en la Figura 2.4 del Capítulo 2. A nivel de arquitectura global, el codificador `AudioSparsifinerEncoder` ocupa el bloque izquierdo del diagrama, produciendo la memoria $\mathbf{M}$ que el decodificador consume mediante atención cruzada. El mecanismo interno del codificador —proyección de entrada, bloques `Sparsifiner` apilados y predictor de máscara— queda descrito con precisión en las ecuaciones de las Secciones 4.2 y 4.3. En síntesis, la arquitectura *Magdalena v1.0* se caracteriza por los siguientes aspectos diferenciales respecto a un Transformer estándar:

- **Codificador asimétrico con atención dispersa:** El codificador aplica atención dispersa instancia-dependiente mediante el mecanismo `Sparsifiner`, reduciendo el coste de atención efectivo de $\mathcal{O}(T^2)$ a $\mathcal{O}(r_{attn} \cdot T^2)$ con $r_{attn} = 0.25$, sin comprometer la capacidad del modelo para capturar dependencias globales seleccionadas dinámicamente.
- **Predicción de máscara de bajo rango:** La selección de conexiones relevantes se realiza mediante una factorización de bajo rango de la matriz de atención con factores de compresión $r_c = 2$ (canal) y $r_N = 64$ (temporal), lo que mantiene el coste del predictor en $\mathcal{O}(T \cdot k)$ con $k = T/r_N \ll T$.
- **Compatibilidad con longitud variable:** La implementación introduce recorte dinámico (*slicing*) de las matrices de proyección del predictor, eliminando la restricción de longitud fija del `Sparsifiner` original y permitiendo procesar fragmentos de audio de duración arbitraria sin necesidad de *padding* hasta T_{max} .
- **Decodificador estándar con atención cruzada:** El decodificador emplea la arquitectura Transformer canónica con autoatención enmascarada y atención cruzada hacia la memoria del codificador, garantizando la coherencia gramatical de la secuencia de *tokens* MIDI generada mediante la máscara causal.

En conjunto, estas decisiones de diseño permiten que *Magdalena v1.0* procese secuencias de audio de hasta 82 segundos con los recursos de cómputo disponibles, algo inviable con un Transformer estándar a igual configuración. La reducción de dimensionalidad respecto a la arquitectura de referencia ($d = 256$ frente a $d = 512$) es una concesión a las

limitaciones de hardware que el mecanismo Sparsifiner compensa parcialmente al reducir el coste cuadrático de la atención. El Capítulo 5 describe los detalles de entrenamiento y la estrategia experimental adoptada para validar estas elecciones dentro de las restricciones existentes.

5. Implementación

El presente capítulo describe las decisiones de diseño e ingeniería adoptadas para llevar a la práctica la arquitectura definida en el Capítulo 4. Se detallan el entorno de ejecución, el flujo de procesamiento de datos, el procedimiento de entrenamiento y la configuración de los experimentos realizados. Finalmente, se describe el sistema de inferencia y evaluación empleado para medir el rendimiento del modelo.

5.1. Entorno de desarrollo

El desarrollo y entrenamiento del sistema se ha llevado a cabo íntegramente en Google Colaboratory (*Colab*), un entorno de computación en la nube que proporciona acceso gratuito o de bajo coste a unidades de procesamiento gráfico (GPU). Los experimentos descritos en este capítulo se ejecutaron sobre una GPU NVIDIA T4 con 15 GB de memoria de vídeo (VRAM). Esta restricción de memoria fue el factor determinante para adoptar la configuración reducida del modelo ($d = 256$, cuatro capas) en lugar de la configuración completa ($d = 512$, seis capas), tal y como se discutió en la Sección 4.1.

La Tabla 5.1 recoge las principales dependencias entre librerías software del proyecto.

Biblioteca	Versión	Uso
Python	3.12	Lenguaje principal.
PyTorch	2.1	Framework de aprendizaje profundo; definición, entrenamiento e inferencia del modelo.
torchaudio	2.1	Procesamiento de tensores de audio.
librosa	0.10	Extracción del espectrograma Log-Mel.
pretty_midi	0.2	Lectura y escritura de archivos MIDI.
music21	9.1	Exportación a formato MusicXML.
mir_eval	0.7	Cálculo de métricas estándar de AMT.
einops	0.7	Manipulación de tensores en el encoder Sparsifiner.
customtkinter	5.2	Interfaz gráfica de la aplicación.
pyyaml	6.0	Gestión de configuraciones por experimento.
numpy	1.24	Operaciones numéricas base en todo el pipeline de audio y entrenamiento.
pandas	2.1	Lectura y filtrado de los metadatos del dataset MAESTRO (CSV).
tensorboard	2.14	Seguimiento de métricas de entrenamiento y evaluación F1.
matplotlib	3.8	Visualización de espectrogramas y piano rolls.

Tabla 5.1: Stack tecnológico del proyecto.

5.2. Pipeline de datos

El flujo de procesamiento de datos transforma los archivos brutos del corpus MAESTRO en tensores listos para el entrenamiento. Este flujo se implementa en tres módulos independientes: `AudioProcessor`, `MidiProcessor` y `MaestroDataset`.

5.2.1. Procesamiento de audio

El módulo `AudioProcessor` encapsula la transformación de la señal de audio al espectrograma Log-Mel descrita en la Sección 3.2, con los parámetros de la Tabla 3.2. Adicionalmente, aplica una normalización en dos pasos: primero convierte las magnitudes de potencia a decibelios mediante `librosa.power_to_db` con referencia al máximo de la señal y un rango dinámico de 60 dB; a continuación, reescala los valores del intervalo $[-60, 0]$ dB

al rango $[0, 1]$ de forma lineal. Esta normalización homogeneiza la entrada entre piezas de distinta intensidad media y estabiliza la distribución de los gradientes durante el entrenamiento.

5.2.2. Tokenización MIDI

El módulo `MidiProcessor` convierte los archivos MIDI del corpus en secuencias de *tokens* discretos siguiendo el vocabulario definido en la Sección 3.3. Los eventos se ordenan cronológicamente y, ante empates temporales, los eventos *Note-Off* reciben prioridad absoluta sobre los *Note-On*, previniendo colisiones lógicas en la representación. La cuantización de la velocidad proyecta los 128 valores MIDI (0–127) en 32 *bins* uniformes, con saturación en el valor máximo para evitar desbordamientos.

5.2.3. Carga y empaquetado de datos

La clase `MaestroDataset` carga los pares (espectrograma, secuencia de *tokens*) a partir del fichero de metadatos oficial de MAESTRO, leído con `pandas` para filtrar y asignar cada interpretación a su partición correspondiente (*train*, *validation* o *test*). Para mantener la viabilidad computacional en el entorno de entrenamiento, se aplica una truncación de la secuencia de audio a un máximo de `MAX_AUDIO_FRAMES` tramas, y de la secuencia MIDI a un máximo de `MAX_MIDI_TOKENS` *tokens*. Los valores concretos de estos límites varían entre experimentos y se detallan en la Sección 5.4. Las secuencias más cortas que la longitud máxima del lote se rellenan con ceros (*padding*) en la función de empaquetado (`collate_fn`), requisito del sistema de *batching* de PyTorch para construir tensores rectangulares. Esto no contradice la capacidad del modelo para tratar secuencias de longitud variable: en el decoder, los tokens de relleno quedan enmascarados mediante `tgt_padding_mask` para que la atención no los considere, y la función de pérdida emplea `ignore_index=0` para que no contribuyan al gradiente. El encoder de audio no aplica máscara explícita sobre los frames paddeados, lo cual es admisible dado que el valor cero en el espectrograma Log-Mel corresponde a ausencia de señal.

5.3. Procedimiento de entrenamiento

5.3.1. Estrategia de entrenamiento del decodificador

Durante el entrenamiento se emplea *teacher forcing* estricto: en cada paso, el decodificador recibe como entrada la secuencia de *tokens* objetivo desplazada una posición hacia la derecha (`batch_midi[:, :-1]`), mientras que los *tokens* objetivo reales (`batch_midi[:, 1:]`) se usan para calcular la pérdida. Esta estrategia acelera la convergencia al evitar la acumulación de errores propia de la decodificación autorregresiva, a costa de introducir una discrepancia entre las condiciones de entrenamiento e inferencia conocida como *exposure bias*: en inferencia el modelo consume sus propias predicciones, no los *tokens* de referencia.

5.3.2. Función de pérdida

La función de pérdida empleada es la entropía cruzada categórica, implementada mediante (`nn.CrossEntropyLoss`) con `ignore_index=0`, de modo que el *token* de relleno `<pad>` no contribuye al gradiente. No se aplica suavizado de etiquetas (*label smoothing*) ni ponderación diferencial por clase.

5.3.3. Optimización

El optimizador empleado es AdamW [Vaswani et al., 2017], con una tasa de aprendizaje constante de $\eta = 1 \times 10^{-4}$, factor de decaimiento de pesos `weight_decay = 0.01` y parámetros de momento por defecto ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$). No se implementó ningún *scheduler* de la tasa de aprendizaje en los experimentos actuales, lo que constituye una limitación reconocida que se discute en el Capítulo (referencia al cap 7 de conclusiones) 7.

5.3.4. Técnicas de estabilización

Para mejorar la estabilidad numérica del entrenamiento se aplican dos mecanismos complementarios. El primero es el recorte de gradientes (*gradient clipping*) con norma máxima $\|\nabla\|_{max} = 1.0$, que previene explosiones de gradiente características de las arquitecturas Transformer en las primeras épocas. El segundo es la precisión mixta (*mixed precision training*) mediante `torch.amp.autocast` y `torch.amp.GradScaler`: los cálculos de propagación hacia adelante se realizan en formato `float16`, reduciendo el consumo de memoria y el tiempo de cómputo, mientras que las actualizaciones de parámetros se efectúan en `float32` para mantener la precisión numérica.

5.3.5. Gestión de checkpoints

El sistema de entrenamiento guarda el estado completo del modelo (*checkpoint*) al final de cada época, incluyendo los pesos del modelo, el estado del optimizador y el estado del `GradScaler`. En cada ejecución, el código detecta automáticamente el *checkpoint* más reciente disponible y reanuda el entrenamiento desde ese punto, lo que permite recuperarse de interrupciones del entorno Colab sin perder el progreso acumulado.

5.4. Diseño experimental

La restricción de 15 GB de VRAM de la GPU T4 impone límites estrictos sobre el tamaño efectivo del lote y la longitud máxima de las secuencias procesables simultáneamente. Para explorar el espacio de configuraciones dentro de estas restricciones, se diseñaron tres experimentos con un eje de variación diferente cada uno, manteniendo fijo el resto de hiperparámetros. En todos los casos se alcanza un lote efectivo de 32 secuencias mediante distintas combinaciones de tamaño de lote real y acumulación de gradientes.

La Tabla 5.2 recoge los parámetros variables de cada configuración. Los hiperparámetros arquitectónicos (Tabla 4.1) y los de optimización (AdamW, $\eta = 10^{-4}$, *gradient clipping*) son idénticos en los tres experimentos.

Parámetro	v1_baseline	exp01	exp02
<i>Eje de variación</i>	Referencia	Tamaño de datos	Longitud de clip
Muestras (train / val)	Todo / Todo	150 / 30	Todo / Todo
MAX_AUDIO_FRAMES	4 096	4 096	2 048
MAX_MIDI_TOKENS	1 500	1 500	750
Duración máxima (audio)	≈82 s	≈82 s	≈41 s
BATCH_SIZE	8	2	4
GRAD_ACCUM_STEPS	4	16	8
Lote efectivo	32	32	32
Épocas configuradas	80	1 000	1 000

Tabla 5.2: Parámetros variables por experimento. El lote efectivo se mantiene constante en 32 en todas las configuraciones mediante distintas combinaciones de BATCH_SIZE y acumulación de gradientes.

- **v1_baseline** constituye la configuración de referencia: emplea la totalidad del corpus de entrenamiento con clips de hasta 82 segundos. La duración de 80 épocas es reducida respecto a los demás experimentos debido al mayor tiempo por época que implica procesar el corpus completo.
- **exp01_reduced_data** reduce drásticamente el volumen de datos a 150 muestras de entrenamiento y 30 de validación. Su objetivo es validar que el modelo es capaz de converger y sobreajustarse a un subconjunto pequeño, lo que actúa como *sanity check* de la implementación antes de escalar al corpus completo.
- **exp02_short_clips** mantiene el corpus completo pero reduce a la mitad la longitud máxima de las secuencias (2 048 tramas de audio y 750 tokens MIDI). Esta reducción permite duplicar el tamaño real del lote (de 2 a 4 secuencias por paso con `grad_accum=8`) dentro de los mismos límites de VRAM, e hipotéticamente facilita el aprendizaje de eventos locales al eliminar del contexto de entrenamiento las secciones de silencio que preceden a muchas piezas.

5.5. Sistema de inferencia y evaluación

5.5.1. Estrategias de decodificación

En inferencia, el sistema soporta dos estrategias de decodificación autorregresiva. La primera es la decodificación voraz (*greedy decoding*): en cada paso se selecciona el *token* con mayor *logit* mediante `argmax` y se incorpora a la secuencia de entrada del siguiente paso. Es determinista y computacionalmente eficiente, pero tiende a generar secuencias repetitivas en tareas con alta entropía de salida.

La segunda estrategia es el muestreo con temperatura (*temperature sampling*): los *logits* se dividen por un factor de temperatura $\tau = 0.8$ antes de aplicar *softmax*, y el siguiente *token* se muestrea de la distribución resultante mediante `torch.multinomial`. Valores de $\tau < 1$ concentran la distribución hacia los *tokens* más probables, reduciendo la aleatoriedad respecto al muestreo uniforme. Ambas estrategias se detienen al predecir el *token* `<eos>` (índice 310) o al alcanzar la longitud máxima de generación (1 050 *tokens*).

5.5.2. Protocolo de evaluación

La evaluación cuantitativa del sistema se realiza mediante la biblioteca `mir_eval`, que implementa las métricas estándar de AMT descritas en la Sección 2.7. Se calculan tres variantes del F1-score a nivel de nota, con las ventanas de tolerancia detalladas en la Tabla 5.3.

Métrica	Criterio de acierto
F1 de <i>onset</i>	Tono correcto + <i>onset</i> dentro de ± 50 ms
F1 de <i>onset</i> + <i>offset</i>	Anterior + <i>offset</i> dentro de ± 50 ms o 20 % de la duración
F1 de <i>onset</i> + velocidad	Anterior (sin <i>offset</i>) + velocidad dentro de ± 10 %

Tabla 5.3: Criterios de acierto para cada variante del F1-score de nivel de nota, implementados con `mir_eval`.

La evaluación se ejecuta automáticamente sobre el conjunto de validación cada 20 épocas durante el entrenamiento, con los resultados registrados en TensorBoard para su seguimiento. La evaluación final sobre el conjunto de prueba se realiza con el modelo correspondiente al *checkpoint* de menor pérdida de validación.

6. Resultados Experimentales

El presente capítulo describe el protocolo de evaluación aplicado, presenta las curvas de entrenamiento obtenidas durante los experimentos y reporta las métricas cuantitativas sobre el conjunto de prueba del corpus MAESTRO. Se analiza a continuación el comportamiento diferencial de los tres modelos entrenados y se contextualiza su rendimiento en relación con el estado del arte.

6.1. Protocolo de evaluación

La evaluación cuantitativa se realizó sobre una muestra aleatoria de 30 piezas del conjunto de prueba oficial de MAESTRO (semilla fija $s = 42$), extraída del total de 125 interpretaciones del *split* de prueba (véase Tabla 3.1). El tamaño muestral se limitó a 30 piezas por las restricciones de tiempo de cómputo del entorno Google Colaboratory; la selección aleatoria con semilla fija garantiza la reproducibilidad del resultado.

Para `v1_baseline` y `exp02_short_clips`, el *checkpoint* evaluado se determinó de forma manual a partir del sufijo de ranking por pérdida de validación que Google Colaboratory añade automáticamente al nombre de los archivos de *checkpoint* guardados (p. ej. `-001`, `-002`, ...); en cada caso se seleccionó el *checkpoint* mejor posicionado en dicho ranking. Para `exp01_reduced_data` este mecanismo de ranking no estuvo disponible, por lo que se empleó directamente el *checkpoint* de la última época del entrenamiento (época 514). En ningún caso se conservó un registro numérico exportable de la pérdida de validación por época, por lo que las curvas presentadas en la Sección 6.2 no incluyen dicha señal; el criterio empleado para seleccionar el *checkpoint* y las métricas F1 analizadas en esa sección proceden, por tanto, de dos fuentes de información distintas.

La estrategia de decodificación empleada en todos los casos fue el muestreo con temperatura $\tau = 0.8$, tal y como se describió en la Sección 5.5.

Las métricas calculadas con `mir_eval` son las tres variantes a nivel de nota definidas en la Sección 2.7: F1 de *onset* (tolerancia ± 50 ms), F1 de *onset + offset* (± 50 ms o 0.2 de la duración) y F1 de *onset + velocidad* (± 50 ms más tolerancia del 0.1 en velocidad).

6.2. Curvas de entrenamiento

Para cada configuración experimental se registró la evolución de la pérdida de entrenamiento y de las métricas F1 sobre el conjunto de validación. Como se ha indicado en la Sección 6.1, el *checkpoint* evaluado en `v1_baseline` y `exp02_short_clips` se seleccionó de forma manual a partir del ranking por pérdida de validación generado automáticamente por Google Colaboratory, y no a partir de las métricas F1 que se presentan a continuación; en `exp01_reduced_data`, al no disponer de dicho ranking, se empleó el *checkpoint* de la última época. Esta distinción es relevante para interpretar correctamente las figuras siguientes: el punto marcado como mejor *checkpoint* no tiene por qué coincidir con el máximo de F1 visible en cada gráfica, puesto que ambas señales –pérdida de validación y F1– se calcularon de forma independiente y no necesariamente evolucionan de forma idéntica.

6.2.1. `v1_baseline`

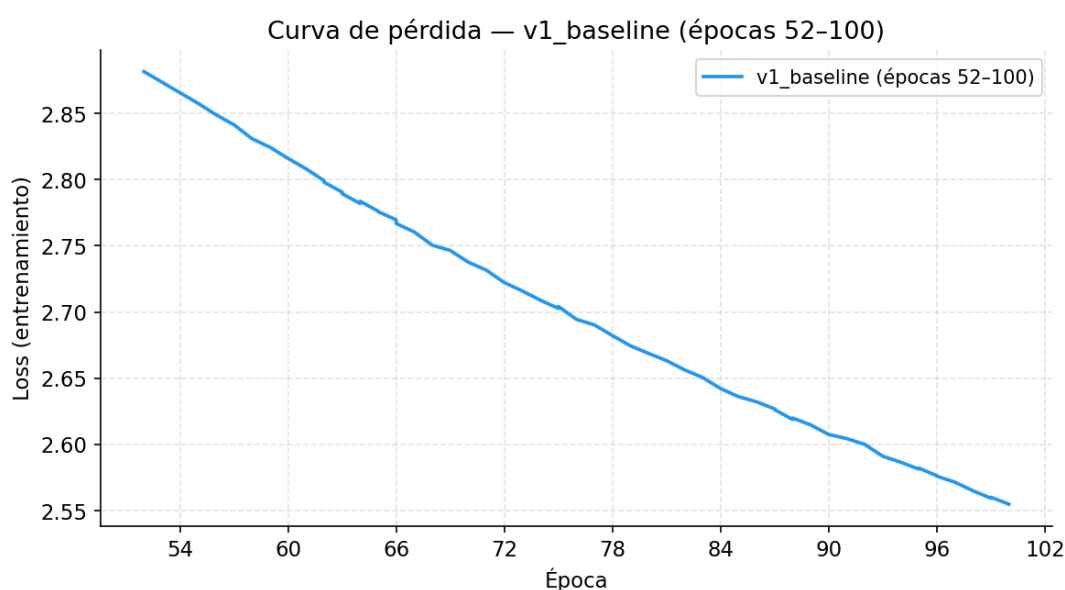


Figura 6.1: Evolución de la pérdida de entrenamiento de `v1_baseline` entre las épocas 52 y 100 (corpus completo, `MAX_AUDIO_FRAMES` = 4 096).

La pérdida de entrenamiento (Figura 6.1) presenta las siguientes características en el tramo disponible:

- **Rango:** desciende de 2.88 (época 52) a aproximadamente 2.56 (época 98-100).
- **Forma:** descenso prácticamente monótono, sin ruido apreciable.
- **Cobertura:** no hay registro de las primeras 52 épocas, por lo que no puede describirse el comportamiento inicial del entrenamiento.

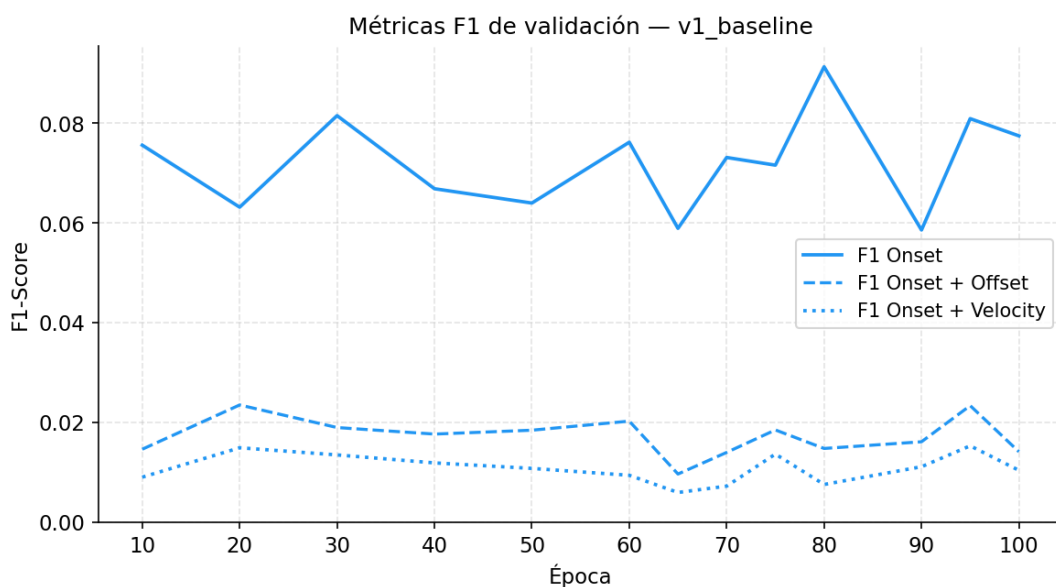


Figura 6.2: Métricas F1 de validación de *v1_baseline*. El mejor *checkpoint* corresponde a la época 98.

La evolución del F1 de validación (Figura 6.2) contrasta con la suavidad de la curva de pérdida:

- **F1 onset:** oscila entre 0.059 y 0.091 sin tendencia de mejora sostenida; máximo en la época 80, mínimos locales en las épocas 65 y 90.
- **F1 onset+offset:** se mantiene en una banda estrecha de 0.010–0.023 durante todo el entrenamiento.
- **F1 onset+velocity:** banda de 0.006–0.015.
- **Checkpoint evaluado (época 98):** seleccionado por su posición en el ranking de pérdida de validación de Colaboratory, no por F1. Los valores vecinos (épocas 95 y 100) son 0.081 y 0.077 respectivamente. Esto representa una franja de F1 alta pero no máxima, ya que el pico de la curva (0.091) se da en la época 80.

Esta discrepancia entre una pérdida de entrenamiento que mejora de forma constante y un F1 de validación mucho más ruidoso sugiere que, en este experimento, la reducción continuada de la pérdida no se traduce necesariamente en una mejora proporcional y estable de la calidad de la transcripción. La cercanía entre el *checkpoint* elegido por pérdida de validación y una zona de F1 razonablemente alta es, sin ser una coincidencia exacta, una señal tranquilizadora de que ambos criterios no divergen de forma severa en este experimento.

6.2.2. exp01_reduced_data

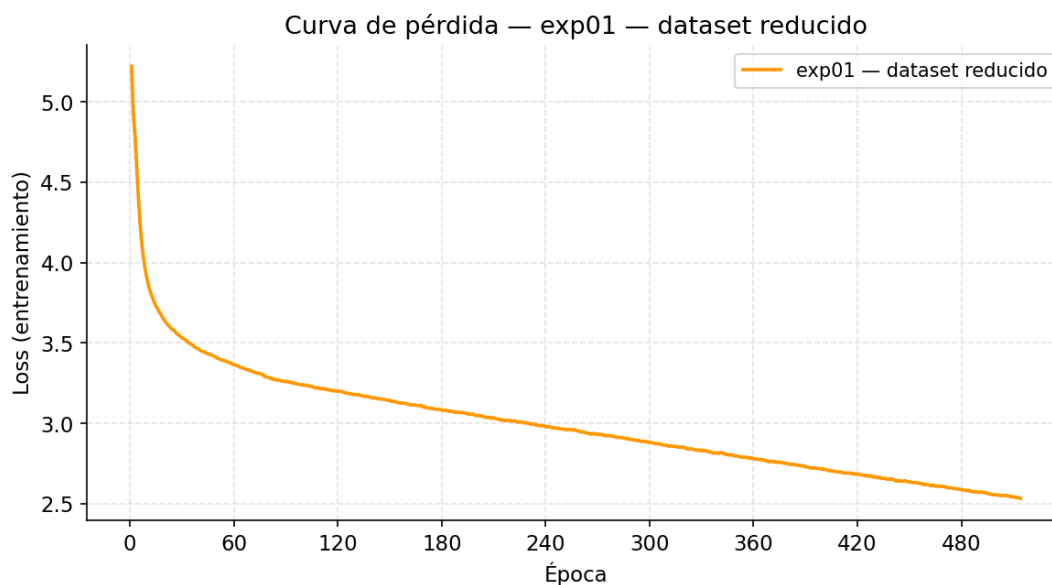


Figura 6.3: Evolución de la pérdida de entrenamiento de exp01_reduced_data (150 muestras de entrenamiento).

El modelo correspondiente a exp01_reduced_data entrenó con solo 150 muestras. Su curva de pérdida de entrenamiento (Figura 6.3) muestra:

- **Rango:** de 5.25 (época 0) a 2.53 (época 514).
- **Caída inicial:** descenso rápido hasta ≈ 3.5 en las primeras 30–40 épocas.
- **Tramo final:** descenso lento y prácticamente lineal durante el resto del entrenamiento, sin señales de estancamiento.

Este descenso sostenido durante más de 500 épocas, típico de un escenario de sobreajuste a un conjunto de datos reducido, no puede contrastarse directamente con una curva de pérdida de validación, al no haberse conservado un registro numérico de esta última.

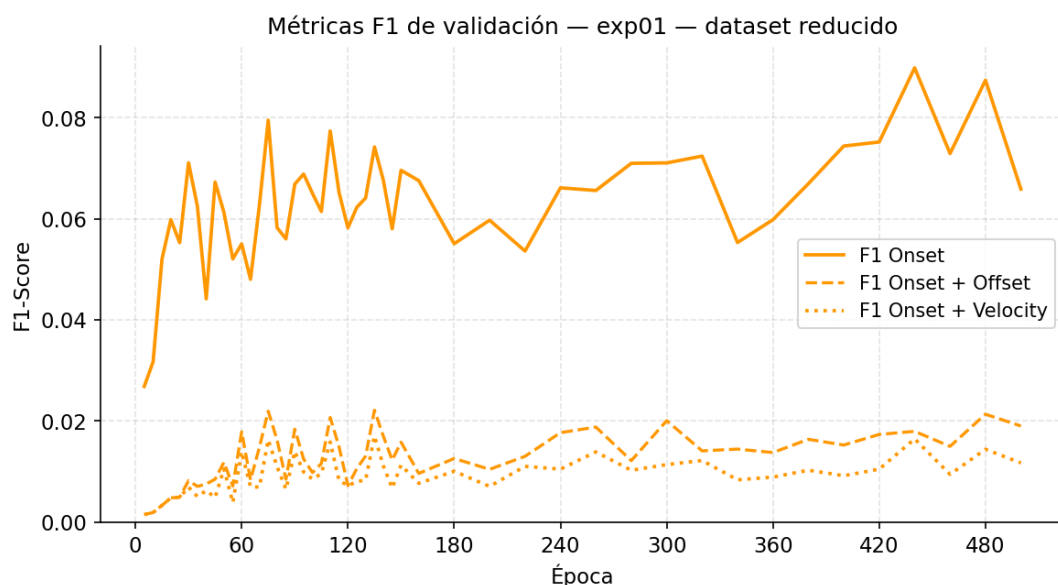


Figura 6.4: Métricas F1 de validación de exp01_reduced_data. Se empleó el *checkpoint* de la última época disponible (514) al no disponer de ranking de pérdida de validación.

Las métricas F1 de validación (Figura 6.4) matizan esta lectura:

- **Subida inicial:** rápida durante las primeras 30–40 épocas.
- **F1 onset:** oscila con varianza considerable en una banda de 0.05–0.09, con ligera tendencia ascendente hacia el final; máximos globales en torno a las épocas 430 y 480 (≈ 0.090 y ≈ 0.087).
- **F1 onset+offset y onset+velocity:** patrón similar, con magnitudes menores.
- **Checkpoint evaluado (época 514):** sin ranking de pérdida de validación disponible, se empleó directamente la última época –el único de los tres experimentos sin ningún criterio de rendimiento asociado. Su F1 onset (≈ 0.066) cae en un valle local, sensiblemente por debajo de los picos observados 30–80 épocas antes.

Esta tendencia no muestra la degradación de F1 que cabría esperar de un sobreajuste severo. El desajuste entre el punto de máximo rendimiento observado durante el entrenamiento y el *checkpoint* efectivamente utilizado constituye una explicación plausible, adicional a la limitada capacidad de generalización propia de un conjunto de entrenamiento tan reducido, de por qué exp01_reduced_data obtiene los resultados más bajos de los tres experimentos sobre el conjunto de *test* (F1 onset de 0.065, con solo 1 de 30 piezas alcanzando un F1 superior a 0.10).

6.2.3. exp02_short_clips

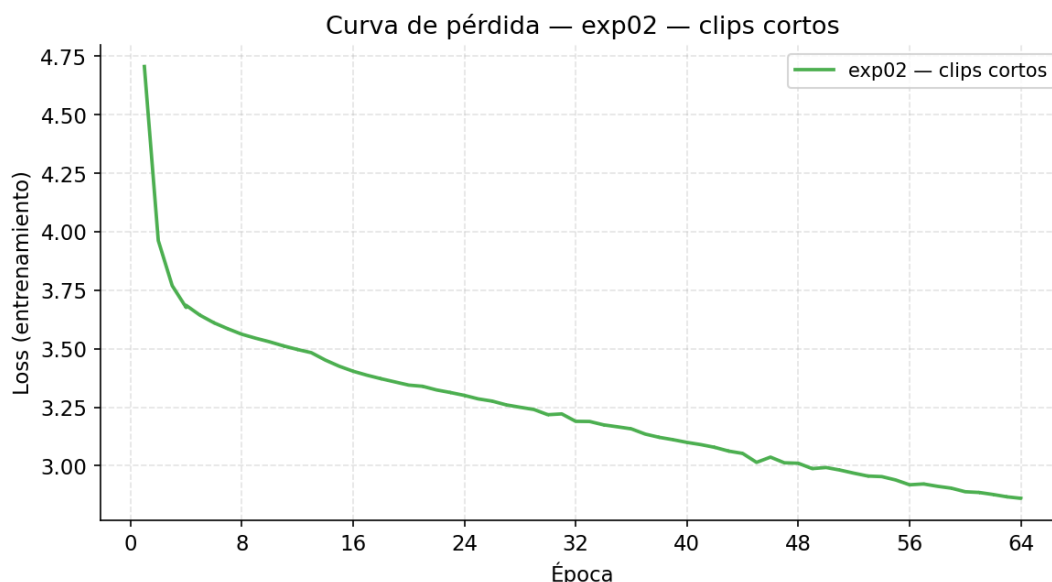


Figura 6.5: Evolución de la pérdida de entrenamiento de exp02_short_clips (corpus completo, MAX_AUDIO_FRAMES = 2 048).

La curva de pérdida de entrenamiento (Figura 6.5) presenta:

- **Rango:** de 4.7 (época 0) a 2.85 (época 64).
- **Caída inicial:** pronunciada, hasta ≈ 3.7 en las primeras 5 épocas.
- **Tramo posterior:** descenso continuo y suave, sin estancamiento ni repuntes en el registro disponible.

El F1 de validación (Figura 6.6) también mejora de forma sostenida, aunque ruidosa:

- **F1 onset:** sube de ≈ 0.06 (épocas iniciales) a un máximo de ≈ 0.082 hacia la época 56.
- **F1 onset+offset:** tendencia ascendente clara en la segunda mitad del entrenamiento, de ≈ 0.008 (época 35) a ≈ 0.021 (época 55).
- **Checkpoint evaluado (época 57):** seleccionado por el ranking de pérdida de validación de Colaboratory, no directamente por F1; cae muy cerca de los valores más altos de F1 observados en el rango registrado.

Ni la pérdida de entrenamiento ni el F1 de validación muestran estancamiento o divergencia en torno a la época 57, ambas señales evolucionan de forma consistente hasta donde alcanza el registro disponible indicando que el entrenamiento se interrumpió mientras el modelo aún tenía capacidad de mejora, y no como consecuencia de una degradación por sobreajuste detectable en estas curvas.

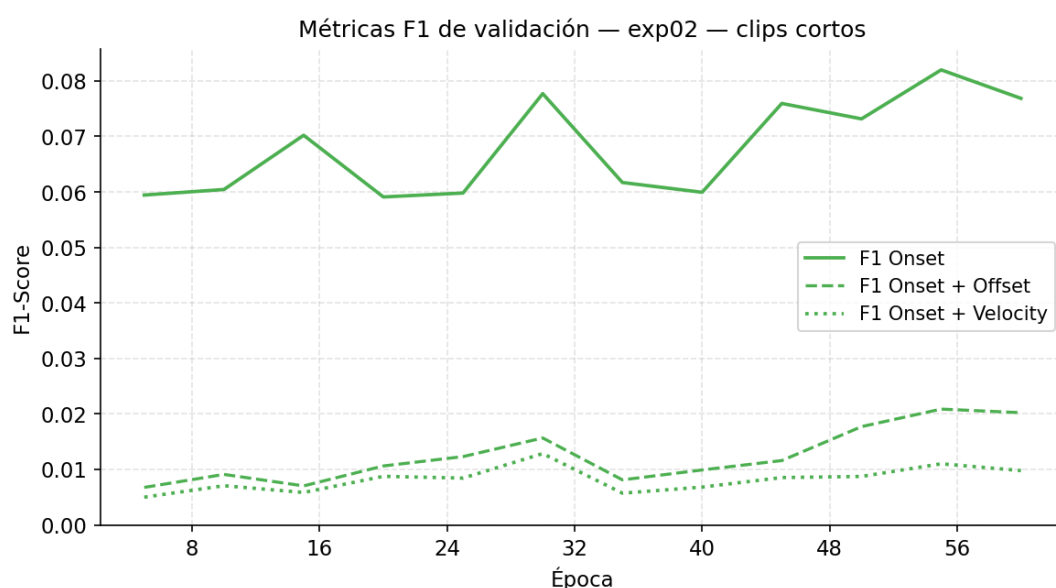


Figura 6.6: Métricas F1 de validación de exp02_short_clips. El mejor *checkpoint* corresponde a la época 57.

6.2.4. Comparativa entre experimentos

Las Figuras 6.7 y 6.8 permiten resumir las diferencias entre experimentos (ver siguiente página):

- **Escala temporal:** v1_baseline y exp02_short_clips completan su entrenamiento en un centenar de épocas; exp01_reduced_data, con muchas menos muestras por época, se extiende durante más de 500.
- **F1 onset de validación:** rango comparable entre los tres experimentos (≈ 0.06 – 0.09) durante sus respectivas fases de entrenamiento.
- **F1 onset+offset y onset+velocity:** muy por debajo del F1 onset en los tres casos, en línea con la caída observada en *test* entre *onset* y las métricas combinadas: 0.665 (v1_baseline), 0.809 (exp01_reduced_data) y 0.750 (exp02_short_clips).
- **Criterio de selección de *checkpoint*:**
 - v1_baseline y exp02_short_clips: ranking automático por pérdida de validación de Colaboratory, razonablemente coherente con las métricas F1 mostradas.
 - exp01_reduced_data: ranking no disponible; se empleó la última época, sin ningún criterio de rendimiento asociado.

Esta ausencia total de un criterio de selección en exp01_reduced_data (y no solo la escasez de datos de entrenamiento) constituye una explicación adicional y bien fundamentada de su resultado comparativamente más bajo sobre el conjunto de *test*.

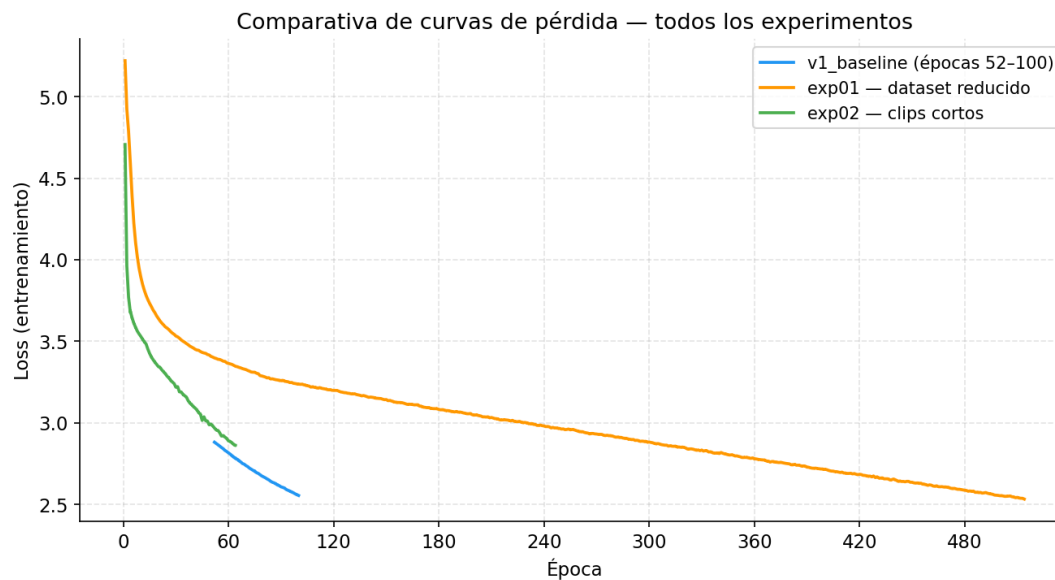


Figura 6.7: Comparativa de las curvas de pérdida de entrenamiento de los tres experimentos.

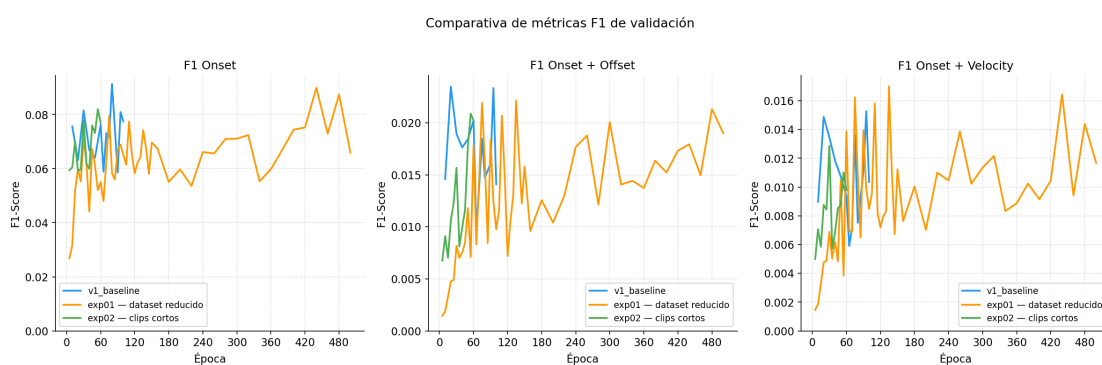


Figura 6.8: Comparativa de las métricas F1 de validación (onset, onset+offset y onset+velocity) de los tres experimentos.

6.3. Análisis cualitativo

La Tabla 6.1 resume las métricas obtenidas por cada modelo sobre las 30 piezas del conjunto de prueba, calculadas a partir de los resultados individuales por pieza con `mir_eval`.

Tabla 6.1: Métricas de evaluación sobre el conjunto de prueba (30 piezas, semilla 42). Los valores se calculan pieza a pieza y se agregan mediante media y desviación típica.

Métrica	v1_baseline	exp01	exp02
F1 onset (media $\pm$ std)	0.080 ± 0.025	0.065 ± 0.018	0.103 ± 0.042
F1 onset (mediana)	0.071	0.064	0.097
F1 onset (rango)	0.041–0.139	0.032–0.115	0.028–0.188
F1 onset+offset (media $\pm$ std)	0.029 ± 0.017	0.013 ± 0.007	0.029 ± 0.022
F1 onset+velocity (media $\pm$ std)	0.016 ± 0.011	0.007 ± 0.005	0.017 ± 0.014
Notas GT (media)	877	877	450
Notas predichas (media)	618	624	543
Ratio pred/GT (media por pieza)	0.79	0.80	1.32
Piezas con F1 onset > 0.10	8/30	1/30	14/30
Piezas con F1 onset > 0.5	0/30	0/30	0/30

6.3.1. Patrón de predicción de notas

Los tres modelos exhiben comportamientos distintos en cuanto al número de notas predichas respecto a la referencia:

- **v1_baseline:** infrapredice de forma consistente, generando en promedio el 0.79 de las notas de referencia (618 de 877 notas).
- **exp01_reduced_data:** presenta un patrón muy similar a **v1_baseline**, con un 0.80 de las notas de referencia (624 de 877).
- **exp02_short_clips:** suprapredice, con un 1.32 de las notas de referencia (543 de 450).

Antes de interpretar esta diferencia, es necesario señalar una precaución metodológica: el promedio de notas GT de **exp02_short_clips** (450) es aproximadamente la mitad del de **v1_baseline** y **exp01_reduced_data** (877 en ambos casos), a pesar de evaluarse sobre las mismas 30 piezas. Esto no se debe a que las piezas sean distintas, sino a que **exp02_short_clips** trunca el audio a `MAX_AUDIO_FRAMES = 2 048` (aproximadamente 41 segundos) frente a los 4 096 (aproximadamente 82 segundos) de los otros dos modelos: la referencia MIDI utilizada en la evaluación se recorta a la misma ventana temporal que ve

el modelo, por lo que `exp02_short_clips` se evalúa, en promedio, sobre menos de la mitad de notas por pieza que los demás. En consecuencia, los recuentos absolutos de notas de `exp02_short_clips` no son directamente comparables con los de los otros dos experimentos; el ratio predicción/referencia calculado *dentro* de la ventana propia de cada modelo sí lo es, y es el que sostiene la lectura anterior: los modelos entrenados sobre clips largos tienden a la cautela, mientras que el entrenamiento sobre clips cortos tiende a sobregenerar notas, posiblemente porque los fragmentos breves de MAESTRO contienen una proporción más alta de pasajes musicalmente activos que las piezas completas, y el modelo aprendió a activar notas con mayor frecuencia en consecuencia.

6.3.2. Brecha entre onset y onset+offset

Los tres modelos exhiben una caída sistemática del F1 al añadir la exigencia de detección de *offset*, calculada como la caída porcentual media pieza a pieza:

- **v1_baseline:** caída del 0.665 (F1 *onset* de 0.080 a F1 *onset+offset* de 0.029).
- **exp02_short_clips:** caída del 0.750 (de 0.103 a 0.029).
- **exp01_reduced_data:** caída del 0.809 la más pronunciada de los tres (de 0.065 a 0.013).

Este patrón indica que el sistema ha aprendido a detectar el inicio de las notas con cierta consistencia, pero prácticamente no modela su duración. Es coherente con las propiedades acústicas del problema: el *onset* produce un transitorio espectral distintivo y bien localizado en el tiempo, mientras que el *offset* depende del decaimiento acústico y del pedal de resonancia; fenómenos de mayor complejidad temporal que requieren capturar dependencias a largo plazo, precisamente donde las restricciones de truncación y tamaño de modelo limitan más al sistema. Que la caída sea máxima en `exp01_reduced_data`, el experimento con menos datos de entrenamiento, es consistente con esta lectura: modelar el *offset* exige generalizar patrones de duración de nota más sutiles que el propio *onset*, y es razonable que sea la capacidad más perjudicada por la escasez de datos.

6.3.3. Efecto de la longitud del clip de entrenamiento

`exp02_short_clips` obtiene el mejor F1 de *onset* medio (0.103) y el mayor número de piezas por encima del umbral de 0.10 (14 de 30), pese a entrenarse con secuencias de la mitad de duración que `v1_baseline`. Dos factores, no excluyentes, pueden explicar esta mejora:

- **Densidad de señal de aprendizaje:** los clips de 41 segundos contienen, en proporción, menos silencio inicial y más eventos musicales activos que las piezas completas de MAESTRO, lo que mejora la densidad de señal de aprendizaje por época.
- **Mayor número de épocas efectivas:** al procesar secuencias más cortas, `exp02_short_clips` completó más pasos de optimización por época que `v1_baseline` en un tiempo de cómputo comparable.

Este beneficio tiene, no obstante, un coste: la incapacidad del modelo para aprender dependencias estructurales a mayor escala temporal, lo que –como se discute en el apartado anterior– puede explicar tanto su mayor caída relativa de F1 al añadir el *offset* respecto a *v1_baseline*, como su tendencia a supraprededir notas al generalizar a piezas completas durante la evaluación, un contexto de duración muy distinto al de entrenamiento.

6.3.4. Varianza entre piezas

Los tres modelos presentan una desviación típica considerable en el F1 de *onset*, especialmente *exp02_short_clips* ($\sigma = 0.042$, frente a $\sigma = 0.025$ en *v1_baseline* y $\sigma = 0.018$ en *exp01_reduced_data*), con un rango de valores que va de 0.028 a 0.188 según la pieza. El cruce de los resultados por pieza entre modelos aporta dos observaciones concretas sobre el origen de esta variabilidad:

- **Dificultad compartida entre configuraciones:** el F1 de *onset* por pieza está correlacionado entre modelos; fuertemente entre *v1_baseline* y *exp02_short_clips* ($r = 0.71$), moderadamente entre *v1_baseline* y *exp01_reduced_data* ($r = 0.38$); lo que indica que una parte de la variabilidad responde a características intrínsecas de cada pieza, no solo a la configuración del modelo. De hecho, la pieza peor transcrita por *v1_baseline* (F1 = 0.041) coincide exactamente con la peor transcrita por *exp01_reduced_data* (F1 = 0.032, identificador 2015/R2_D2-12-13-15), y la mejor pieza de *exp01_reduced_data* (F1 = 0.115) coincide con la mejor de *exp02_short_clips* (F1 = 0.188, identificador 2004/SMF_17_R1_2004).
- **Relación con el número de notas y con el ratio de predicción:** el número de notas GT de la pieza correlaciona de forma débil a moderada y *positiva* con el F1 de *onset* en los tres modelos ($r = 0.28$ en *v1_baseline*, $r = 0.45$ en *exp01_reduced_data*, $r = 0.59$ en *exp02_short_clips*), lo que no respalda la hipótesis de que una mayor densidad de notas perjudique por sí sola el rendimiento. En cambio, la desviación del ratio predicción/GT respecto a 1 sí correlaciona negativamente con el F1 en los tres casos (r entre -0.30 y -0.53): las piezas en las que el modelo predice un número de notas más alejado de la referencia son sistemáticamente las peor transcritas.

Estos resultados sugieren que la varianza entre piezas responde en buena medida a una combinación de dificultad intrínseca (parcialmente compartida entre configuraciones) y de la capacidad del modelo para calibrar el número de notas que genera, más que a la densidad polifónica por sí sola. No se dispone de metadatos por pieza (tempo, uso del pedal de resonancia, género) que permitan contrastar directamente otras hipótesis explicativas, lo que se señala como limitación en la Sección 6.4.3.

6.4. Discusión

6.4.1. Rendimiento en perspectiva

Los valores de F1 de *onset* obtenidos (0.065–0.103) se sitúan por debajo de los publicados por los sistemas del estado del arte. A modo de referencia, Kong et al. [Kong et al., 2021] reportan un F1 de *onset* superior al 0,96 sobre el mismo corpus MAESTRO. Esta brecha debe contextualizarse en función de las diferencias de escala y recursos entre ambos trabajos.

El sistema Magdalena v1.0 fue entrenado con una GPU de 15 GB de VRAM (NVIDIA T4), lo que impuso una configuración reducida ($d = 256$, cuatro capas) frente a las arquitecturas de referencia, sin *scheduler* de la tasa de aprendizaje, sin aumento de datos, con truncación dura a 41–82 segundos y con un número de épocas limitado por el entorno Colaboratory. En este contexto, los resultados no pretenden competir con el estado del arte, sino demostrar que la arquitectura Encoder-Decoder con atención dispersa Sparsifiner es viable en el dominio del audio musical y aprende representaciones con valor predictivo no trivial:

- Los tres modelos producen F1 de *onset* consistentemente superior a cero en la totalidad de las 30 piezas evaluadas, confirmando que el sistema captura patrones acústicos asociados a eventos musicales y no genera transcripciones aleatorias.
- Ninguno de los tres modelos supera un F1 de *onset* de 0.5 en ninguna pieza, lo que confirma que, aun aprendiendo patrones no triviales, el sistema está lejos de una transcripción de calidad práctica en su configuración actual.

6.4.2. Interpretación experimental

La superioridad de `exp02_short_clips` sobre `v1_baseline` en F1 de *onset* sugiere que la longitud del clip de entrenamiento es un hiperparámetro relevante que merece exploración sistemática en trabajos futuros.

La clara inferioridad de `exp01_reduced_data` se explica por dos factores combinados: la escasez de datos de entrenamiento (150 muestras), que limita por sí sola la capacidad de generalización del modelo –y que además se refleja en la mayor caída relativa de F1 al incorporar el *offset* (0.809, la más alta de los tres experimentos)–, y la ausencia de cualquier criterio de selección de *checkpoint* –ni por ranking de pérdida de validación ni por F1–, que obligó a evaluar el *checkpoint* de la última época disponible sin garantía de que fuera el de mejor rendimiento. Pese a ello, el análisis por pieza muestra que `exp01_reduced_data` no es un modelo que produzca resultados aleatorios: su F1 por pieza correlaciona de forma moderada con el de `v1_baseline` ($r = 0.38$) y comparte con él la misma pieza como la peor transcrita, lo que indica que, con datos muy limitados, el modelo aprendió una señal débil pero genuina, coherente con la de los otros experimentos. Este experimento cumplió, en todo caso, su función como *sanity check* de la implementación.

La convergencia temprana de `exp02_short_clips` (mejor *checkpoint* en la época 57 frente a 98 de `v1_baseline`) no responde, según lo observado en la Sección 6.2, a una degradación por sobreajuste: ni la pérdida de entrenamiento ni el F1 de validación muestran divergencia o estancamiento en torno a esa época, y el *checkpoint* evaluado se seleccionó, como en `v1_baseline`, por su posición en el ranking de pérdida de validación de Colaboratory, no por una caída de rendimiento. Una explicación más plausible es que, al operar con clips de menor duración (41 segundos frente a los 82 segundos de `v1_baseline`), la dinámica de optimización por época difiere sustancialmente entre ambos experimentos, lo que puede desplazar el punto de mejor pérdida de validación a una época distinta sin que ello implique sobreajuste. Confirmar esta hipótesis, así como determinar si el entrenamiento se interrumpió antes de alcanzar su verdadero óptimo, requeriría extender el registro más allá de la época 64 actualmente disponible. En cualquier caso, esta interpretación no excluye el interés de incorporar aumento de datos o activar el mecanismo *DropPath* en experimentos futuros, no como corrección de un sobreajuste detectado sino como medida preventiva ante entrenamientos más largos o modelos de mayor capacidad.

6.4.3. Limitaciones de la evaluación

La evaluación presenta las siguientes limitaciones metodológicas:

- **Tamaño muestral:** se evaluó sobre 30 piezas en lugar de las 125 del conjunto de prueba completo, lo que introduce variabilidad muestral en las estimaciones. Las desviaciones típicas elevadas, especialmente en `exp02_short_clips` ($\sigma = 0.042$), reflejan esta variabilidad y aconsejan interpretar las medias con cautela.
- **Ventanas de evaluación no equivalentes:** como se ha señalado en la Sección 6.3, la referencia MIDI empleada para evaluar `exp02_short_clips` corresponde a una ventana temporal aproximadamente la mitad de larga que la empleada para `v1_baseline` y `exp01_reduced_data`, al truncarse a `MAX_AUDIO_FRAMES = 2 048` frente a 4 096. Esto no invalida la comparación de las métricas F1 –calculadas siempre dentro de la ventana propia de cada modelo–, pero sí implica que los recuentos absolutos de notas y los patrones de predicción no son directamente comparables entre `exp02_short_clips` y los otros dos experimentos sin tener en cuenta esta diferencia.
- **Ausencia de metadatos por pieza:** no se dispuso de información sistemática sobre tempo, densidad polifónica, uso del pedal de resonancia o género de cada pieza, lo que limita la capacidad de explicar la varianza entre piezas más allá de las correlaciones observadas con el número de notas de referencia y con el ratio de predicción.
- **Registro de pérdida de validación:** al no exportarse un log numérico de la pérdida de validación por época, no es posible verificar a posteriori si el *checkpoint* señalado por el ranking de Colaboratory en `v1_baseline` y `exp02_short_clips` corresponde efectivamente al mínimo global de dicha pérdida, ni reconstruir con precisión por qué dicho ranking no estuvo disponible en `exp01_reduced_data`.

Una evaluación sobre el conjunto de prueba completo, junto con el registro de metadatos por pieza, ofrecería estimaciones más robustas y permitiría contrastar de forma más

rigurosa las hipótesis planteadas sobre el origen de la variabilidad observada.

7. Conclusiones

El presente capítulo sintetiza el trabajo realizado, enumera las contribuciones concretas del proyecto, reconoce sus limitaciones y propone líneas de investigación y desarrollo para continuar el trabajo iniciado.

7.1. Resumen del trabajo

Problema y motivación. La AMT de piano exige convertir audio en una representación simbólica precisa: qué notas suenan, cuándo empiezan y terminan, y con qué intensidad. Su naturaleza polifónica la convierte en una tarea especialmente compleja, con aplicaciones de interés en la educación musical, el análisis musicológico y la preservación del patrimonio sonoro [Benetos et al., 2019, Müller, 2021]. Este trabajo aborda el problema desde una perspectiva poco explorada en la literatura: formular la transcripción como traducción de secuencia a secuencia, en lugar de clasificación por tramas.

Sistema desarrollado. Se implementó *Magdalena v1.0*, un sistema *Encoder-Decoder* asimétrico entrenado sobre MAESTRO:

- **Codificador** (*AudioSparsifierEncoder*): adapta la atención dispersa de Wei et al. [Wei et al., 2023] a audio de longitud variable (Sección 7.2).
- **Decodificador** (*MidiDecoder*): nn.TransformerDecoder estándar, vocabulario propio de 311 *tokens* [Oore et al., 2020].
- **Tres experimentos:** *v1_baseline* (corpus completo), *exp01_reduced_data* (150 piezas) y *exp02_short_clips* (corpus completo, clips de la mitad de duración).

Ante la ausencia de un registro exportable de pérdida de validación, el *checkpoint* evaluado se seleccionó de forma manual mediante el ranking interno de Colaboratory (*v1_baseline*, *exp02_short_clips*) o, cuando este no estuvo disponible, mediante la última época (*exp01_reduced_data*).

Resultados. Sobre el conjunto de prueba (Tabla 6.1):

- F1 *onset*: 0.103 (exp02) > 0.080 (v1_baseline) > 0.065 (exp01); ningún modelo supera F1 = 0.5 en ninguna pieza.
- Caída sistemática de F1 al añadir *offset* (0.665–0.809): el sistema localiza bien el *onset* pero apenas modela la duración de la nota.
- Parte de la dificultad es intrínseca a cada pieza y se comparte entre configuraciones; el desajuste entre notas predichas y de referencia correlaciona negativamente con el F1 en los tres modelos.
- exp01_reduced_data confirma que 150 muestras son insuficientes, agravado por ser el único experimento sin criterio de selección de *checkpoint*.
- La convergencia temprana de exp02_short_clips no muestra evidencia de sobreajuste en las curvas registradas.

En conjunto, los resultados no compiten con el estado del arte –algo esperable dadas las restricciones de escala y cómputo– pero validan la viabilidad de la arquitectura Sparsifiner en el dominio del audio musical, y delimitan con precisión los factores sobre los que debe incidir el trabajo futuro.

7.2. Contribuciones

El presente proyecto ha realizado tres contribuciones diferenciadas al campo de la transcripción automática de música para piano.

La primera es la **formulación integral de la AMT de piano como un problema de traducción de secuencia a secuencia**. A diferencia de los enfoques clásicos basados en clasificación por tramas que tratan el tono, la duración y la dinámica de forma desacoplada, el sistema desarrollado establece un mapeo directo y conjunto entre la representación acústica continua y una secuencia simbólica de eventos musicales. Para ello se diseñó un vocabulario propio de 311 *tokens* que codifica de manera integral los cuatro atributos fundamentales de cada nota (tono, inicio, duración y dinámica), siguiendo la filosofía de representación por eventos de Oore et al. [Oore et al., 2020] con adaptaciones específicas en la cuantización de la velocidad (32 *bins*) y en la resolución temporal (pasos de 10 ms).

La segunda contribución es la **adaptación del mecanismo de atención dispersa Sparsifiner al dominio del audio de longitud variable**. La arquitectura original de Wei et al. [Wei et al., 2023] fue diseñada para procesar imágenes de dimensiones fijas, lo que la hace incompatible directamente con secuencias de audio de duración arbitraria. Para superar esta limitación se introdujeron tres modificaciones de ingeniería sobre la implementación original:

- **Recorte dinámico de la matriz de proyección temporal (*proj_n*)**: en lugar de mantener una matriz de parámetros de tamaño fijo $\mathbf{P}_N \in \mathbb{R}^{T_{max} \times k}$ y aplicarla sobre secuencias rellenas con *padding*, se recorta dinámicamente a la longitud real de cada secuencia en cada paso de inferencia, eliminando el coste computacional y de memoria asociado al relleno.
- **Ajuste dinámico del presupuesto de atención**: la operación `real_k = min(self.attn_budget, N)` garantiza que el presupuesto de conexiones activas nunca supere la longitud real de la secuencia, previniendo errores de índice en secuencias más cortas que la longitud máxima T_{max} .
- **Eliminación de la restricción de longitud fija**: se reemplazó la comprobación de longitud exacta por una cota superior la comprobación. La condición `assert num_tokens == N` del código original, forzaba una longitud de entrada fija e incompatible con el procesamiento de fragmentos de audio de duración variable.

Estas tres modificaciones, tomadas en conjunto, permiten que el codificador procese secuencias de cualquier longitud hasta T_{max} sin necesidad de relleno hasta esa cota, lo que resulta especialmente relevante para un corpus de duración variable como MAESTRO.

La tercera contribución es el **desarrollo de un sistema de transcripción funcional de extremo a extremo**, incluyendo el *pipeline* completo de datos (preprocesamiento acústico, tokenización MIDI y carga por lotes), el ciclo de entrenamiento con precisión mixta y reanudación automática desde *checkpoint*, y el protocolo de evaluación con `mir_eval` sobre el conjunto de prueba oficial de MAESTRO. Este sistema constituye una base empírica reproducible sobre la que contrastar el impacto del mecanismo Sparsifiner frente a la atención densa estándar en el dominio del audio musical.

7.3. Competencias desarrolladas

En cuanto a las competencias enumeradas en la sección 1.3 referentes a la especialidad de Computación, han sido trabajadas en los siguientes aspectos del proyecto desarrollado:

[CM3]. El diseño *Encoder-Decoder asimétrico* de *Magdalena v1.0* es en sí mismo una decisión motivada por un análisis de complejidad: la atención estándar tiene un coste $O(N^2)$ en la longitud de la secuencia, inasumible para secuencias de audio de hasta 70 000 tramas, pero perfectamente manejable para las secuencias de *tokens* MIDI, mucho más cortas, que procesa el decodificador. Esto llevó a reservar la atención dispersa *Sparsifiner* –con coste sublineal en el número de conexiones activas– exclusivamente para el codificador, y a mantener atención densa estándar donde la longitud de secuencia no lo justifica. Sobre esa estrategia se implementaron además las tres adaptaciones de ingeniería descritas en la Sección 7.2 (recorte dinámico de *proj_n*, ajuste dinámico del presupuesto de atención y eliminación de la restricción de longitud fija), todas ellas orientadas a garantizar el rendimiento del sistema dentro de las restricciones de memoria de la GPU disponible (Sección 7.4).

[CM4]. El proyecto recorre el ciclo completo de análisis, diseño y construcción de un sistema inteligente: desde la revisión de los fundamentos del Transformer y los paradigmas de traducción de secuencia a secuencia (Capítulo 2), pasando por el diseño razonado de la arquitectura *Magdalena v1.0* (Capítulo 4) –incluyendo decisiones como el uso de codificación posicional aprendida en el codificador, motivada por la naturaleza bidimensional del espectrograma siguiendo a Dosovitskiy et al. [Dosovitskiy et al., 2021], frente a codificación sinusoidal fija en el decodificador para favorecer la generalización a longitudes no vistas–, hasta su implementación funcional de extremo a extremo y su evaluación empírica (Capítulo 6).

[CM5]. El vocabulario propio de 311 *tokens* (Sección 7.2) constituye una formalización explícita de conocimiento musical humano –tono, *onset*, *offset* y dinámica de cada nota– en una representación discreta y computable, adaptando la filosofía de representación por eventos de Oore et al. [Oore et al., 2020] mediante decisiones de cuantización propias (32 *bins* de velocidad, resolución temporal de 10 ms). En el otro extremo del sistema, la representación de espectrograma Log-Mel formaliza de forma análoga la señal acústica en bruto en una representación computable adecuada para el aprendizaje, cubriendo así el aspecto de *percepción* de la competencia. El aspecto de *actuación* se aborda de forma más limitada, al tratarse de una tarea de transcripción –percepción hacia símbolo– y no de un agente que opere sobre un entorno; la generación autorregresiva de la secuencia de salida es, en todo caso, la forma en que el sistema materializa ese conocimiento formalizado en una decisión computable, símbolo a símbolo.

[CM7]. Todo el proyecto gira en torno al diseño, desarrollo y evaluación de un sistema de aprendizaje computacional: el *pipeline* de entrenamiento con precisión mixta, *teacher forcing* y reanudación automática desde *checkpoint* (Sección 7.2); el diseño de tres experimentos controlados que varían sistemáticamente el volumen de datos y la longitud de secuencia (*v1_baseline*, *exp01_reduced_data*, *exp02_short_clips*); y un protocolo de evaluación cuantitativa con *mir_eval* (Capítulo 6). El corpus MAESTRO, con aproximadamente 200 horas de grabaciones, constituye precisamente el gran volumen de datos del que el sistema extrae de forma automática –sin reglas ni heurísticas hechas a mano– el co-

nocimiento simbólico sobre los eventos musicales que contiene.

7.4. Limitaciones

El sistema desarrollado presenta las siguientes limitaciones:

- **Restricciones de hardware.** Entrenamiento en Google Colaboratory con una GPU T4 de 15 GB, lo que impuso una configuración reducida ($d = 256$, cuatro capas) frente a la de referencia del Transformer ($d = 512$, seis capas), dificultando la comparación directa con el estado del arte.
- **Truncación de secuencia.** Audio limitado a 4 096 tramas (≈ 82 s) en `v1_baseline` y `exp01_reduced_data`, y a 2 048 tramas (≈ 41 s) en `exp02_short_clips`, descartando dependencias musicales a largo plazo en piezas más extensas.
- **Sin scheduler de tasa de aprendizaje.** Tasa constante $\eta = 10^{-4}$, sin *warmup* ni decaimiento coseno [Vaswani et al., 2017], pudiendo limitar la velocidad de convergencia y la calidad del mínimo alcanzado.
- **Sin aumento de datos.** Entrenamiento sobre el audio original de MAESTRO, sin *pitch shifting*, *time stretching* ni ruido añadido, reduciendo la variabilidad efectiva del conjunto de entrenamiento.
- **Teacher forcing sin curriculum learning.** El uso estricto de *teacher forcing* introduce *exposure bias* entre entrenamiento e inferencia; no se implementaron mecanismos como *scheduled sampling*.
- **DropPath inactivo.** Implementado en la arquitectura pero configurado con tasa cero (`drop_path_rate = 0.0`) en todos los experimentos, actuando como función identidad.
- **Selección manual de checkpoint.** Sin registro exportable de pérdida de validación, la selección se basó en el ranking interno no persistente de Colaboratory; en `exp01_reduced_data`, no disponible, se empleó la última época sin ningún criterio de rendimiento, penalizando probablemente su resultado final. Las métricas F1 muestreadas periódicamente presentan además una variabilidad considerable en los tres experimentos.
- **Evaluación reducida y con ventanas no equivalentes.** Evaluación sobre 30 de las 125 piezas de prueba, sin metadatos por pieza (tempo, densidad polifónica, pedal). Además, la ventana de referencia de `exp02_short_clips` es aproximadamente la mitad de larga que la de los otros dos experimentos, por lo que los recuentos absolutos de notas –aunque no las métricas F1– no son directamente comparables entre `exp02_short_clips` y el resto.
- **Especialización en piano.** Vocabulario de 88 notas del rango del piano estándar; la extensión a otros instrumentos requeriría rediseñar el vocabulario y reentrenar desde cero.

7.5. Trabajo futuro

Las limitaciones anteriores trazan las principales líneas de trabajo futuro, ordenadas por impacto esperado sobre el rendimiento del sistema:

- **Configuración completa.** Entrenar con $d = 512$, $H = 8$, $L_{enc} = L_{dec} = 6$ sobre hardware dedicado, permitiendo comparación directa con el estado del arte y midiendo cuánto de la brecha se debe a la restricción de escala.
- **Registro y selección automática de *checkpoint*.** Exportar la pérdida de validación por época y automatizar la selección del mejor *checkpoint* según una métrica objetivo (p. ej. F1 onset), evitando el procedimiento manual actual.
- **Scheduler de tasa de aprendizaje.** Incorporar calentamiento lineal y decaimiento coseno [Vaswani et al., 2017], de alto impacto y bajo coste de implementación.
- **Aumento de datos.** Transposición tonal (± 6 semitonos), estiramiento temporal y ruido de fondo, para ampliar la variabilidad efectiva del corpus y mejorar la generalización.
- **Exploración sistemática de la longitud del clip.** Evaluar distintas longitudes de forma controlada y sobre ventanas de referencia equivalentes, dado que exp02_short_clips obtuvo el mejor resultado global.
- **Decodificación avanzada.** *Beam search*, muestreo con núcleo o temperatura adaptativa, para reducir el *exposure bias* de la decodificación voraz.
- **Chunking solapado.** Sustituir la truncación dura por fragmentación con solapamiento, procesando piezas de duración arbitraria sin descartar información musical.
- **DropPath y ablación.** Explorar tasas no nulas ($p \in [0.1, 0.3]$) en configuraciones más profundas, junto con experimentos de ablación sobre cada componente del Sparsifier.
- **Evaluación ampliada con metadatos.** Extender la evaluación a las 125 piezas del conjunto de prueba y registrar metadatos por pieza (tempo, densidad polifónica, pedal) para contrastar las hipótesis sobre la varianza de rendimiento observada.
- **Generalización a otros instrumentos.** Adaptar el vocabulario al rango tonal de instrumentos polifónicos como la guitarra o el bajo eléctrico, con un corpus equivalente a MAESTRO en calidad de alineamiento.
- **Interfaz gráfica y despliegue.** Desarrollo de la interfaz prevista en el diseño original, y optimización para inferencia en tiempo real mediante cuantización o destilación.

Referencia bibliográfica

- [Ba et al., 2016] Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.
- [Beltagy et al., 2020] Beltagy, I., Peters, M. E., and Cohan, A. (2020). Longformer: The long-document transformer. In *arXiv preprint arXiv:2004.05150*.
- [Benetos et al., 2019] Benetos, E., Dixon, S., Duan, Z., and Ewert, S. (2019). Automatic music transcription: An overview. *IEEE Signal Processing Magazine*, 36(1):20–30.
- [Child et al., 2019] Child, R., Gray, S., Radford, A., and Sutskever, I. (2019). Generating long sequences with sparse transformers. In *arXiv preprint arXiv:1904.10509*.
- [Cont, 2010] Cont, A. (2010). A coupled duration-focused architecture for real-time music-to-score alignment. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(6):974–987.
- [Dosovitskiy et al., 2021] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*.
- [Harris, 1978] Harris, F. J. (1978). On the use of windows for harmonic analysis with the discrete fourier transform. *Proceedings of the IEEE*, 66(1):51–83.
- [Hawthorne et al., 2019] Hawthorne, C., Stasyuk, A., Roberts, A., Simon, I., Huang, C.-Z. A., Dieleman, S., Elsen, E., Engel, J., and Eck, D. (2019). Enabling factorized piano music modeling and generation with the MAESTRO dataset. In *International Conference on Learning Representations*.
- [Hendrycks and Gimpel, 2016] Hendrycks, D. and Gimpel, K. (2016). Gaussian error linear units (GELUs). In *arXiv preprint arXiv:1606.08415*.

-
- [Huang et al., 2016] Huang, G., Sun, Y., Liu, Z., Sedra, D., and Weinberger, K. Q. (2016). Deep networks with stochastic depth. In *European Conference on Computer Vision (ECCV)*, pages 646–661. Springer.
- [Kong et al., 2021] Kong, Q., Li, B., Chen, X., and Wang, Y. (2021). High-resolution piano transcription with pedals by regressing onset and offset times. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 391–395. IEEE.
- [Müller, 2021] Müller, M. (2021). *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Springer, Cham, Switzerland, 2nd edition.
- [Oore et al., 2020] Oore, S., Simon, I., Dieleman, S., Eck, D., and Simonyan, K. (2020). This time with feeling: learning expressive musical performance. *Neural Computing and Applications*, 32(4):955–967.
- [Purwins et al., 2019] Purwins, H., Li, B., Virtanen, T., Schlüter, J., Chang, S.-Y., and Sainath, T. (2019). Deep learning for audio signal processing. *IEEE Journal of Selected Topics in Signal Processing*, 13(2):206–219.
- [Shannon, 1949] Shannon, C. E. (1949). Communication in the presence of noise. *Proceedings of the IRE*, 37(1):10–21.
- [Sigtia et al., 2016] Sigtia, S., Benetos, E., and Dixon, S. (2016). An end-to-end neural network for polyphonic piano music transcription. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 24(5):927–939.
- [Tay et al., 2022] Tay, Y., Dehghani, M., Bahri, D., and Metzler, D. (2022). Efficient transformers: A survey. *ACM Computing Surveys*, 55(6):1–28.
- [Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30.
- [Wei et al., 2023] Wei, C., Chen, L., Wang, Z., Hu, J., Lu, Y., Chen, G., and Zhong, Y. (2023). Sparsifiner: Learning sparse instance-dependent attention for efficient vision transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6538–6547.
- [Xiong et al., 2020] Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., and Liu, T.-Y. (2020). On layer normalization in the transformer architecture. In *International Conference on Machine Learning (ICML)*, pages 10524–10533. PMLR.
- [Zaheer et al., 2020] Zaheer, M., Guruganesh, G., Dubey, K. A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., and Ahmed, A. (2020). BigBird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 17283–17297.
-

Declaración de uso de Inteligencia Artificial

En la elaboración de este Trabajo Fin de Grado se han empleado herramientas de Inteligencia Artificial Generativa como apoyo en tareas de redacción académica, análisis de datos experimentales y revisión técnica de coherencia entre capítulos de la memoria.

Todo el contenido generado con asistencia de estas herramientas ha sido revisado, verificado y, en su caso, corregido por la autora, quien asume la responsabilidad completa sobre la exactitud, el contenido y las conclusiones del presente trabajo. El diseño experimental, la implementación del sistema, la generación de los resultados y su interpretación crítica son responsabilidad exclusiva de la autora.